

FPGA implementation of an AHB to APB Bridge using Verilog HDL

**A
Project Report
Submitted by**

**Ponnitivalasa Devraj
University Roll.No. -22ECE050
Tutika Durga Prasad
University Roll.No. -22ECE056
Tamada Lallu Kumar
University Roll.No. -22ECE058
Shubham Satapathy
University Roll.No. -22ECE062**

**in the partial fulfillment for the award of the degree
of
Bachelor of Technology
in
Electronics and Communication Engineering
Under the esteemed guidance of**

**Dr. Jitendra Kumar
Asst. prof, Dept. of ECE
at**



**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING
SCHOOL OF ENGINEERING
GIET UNIVERSITY, GUNUPUR – 765022
2025-26**

DECLARATION

We hereby declare that the project entitled “FPGA implementation of an AHB to APB Bridge using Verilog HDL” submitted for the B.Tech. Degree is our original work and the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles.

Signature of the Students:

Ponnitivalasa Devraj

Tutika Durga Prasad

Tamada Lallu Kumar

Shubham Satapathy

Place: Gunupur

Date:



**Gandhi Institute of Engineering and Technology
University**

Gunupur – 765022, Dist: Rayagada (Odisha), India

*(Approved by AICTE, Govt. of Orissa and Affiliated to Biju
Patnaik University of Technology)*

Phone: 06857 – 250172(Office), 251156(Principal),
250232(Fax), E-mail: gandhi_giet@yahoo.com

Visit us at www.giet.org

**ISO 9001:2000
Certified Institute**



**DEPARTMENT OF ELECTRONICS AND
COMMUNICATION ENGINEERING**

CERTIFICATE

*This is to certify that the project entitled “**FPGA implementation of an AHB to APB Bridge using Verilog HDL**” is bonafide work carried out by **Ponnitivalasa Devraj, Tutika Durga Prasad, Tamada Lallu Kumar, Shubham Satapathy** bearing University Regd. No.-22UG011329, Regd. No.-22UG011335, Regd. No.-22UG011337, Regd. No.-22UG011341 students of **B. Tech (ECE), GIET University, Gunupur**, during the academic year 2025-26 in partial fulfillment of the requirements for the award of the Degree of **Bachelor of Technology in Electronics and Communication Engineering**.*

Dr. Jitendra Kumar

Project Guide

Assistant Professor, Department
of ECE

Gandhi Institute of Engineering
and Technology University,
Gunupur

External Examiner

Dr. Bibhu Prasad

HoD

Department of ECE

Gandhi Institute of Engineering
and Technology University,
Gunupur

ACKNOWLEDGEMENT

We are profoundly grateful to everyone who has supported and contributed to the "AHB to APB Bridge project. It is our pleasure to express our heartfelt thanks to those directly or indirectly involved in this work. Firstly, we extend our deepest gratitude to our project guide, **Dr. Jitendra Kumar**, Asst. Professor in the Department of Electronics and Communication Engineering, GIET University, Gunupur. His continuous support, insightful guidance, and encouragement have been invaluable throughout the project. Dr. Kumar's belief in our abilities and his efforts to instill confidence in us have been crucial in overcoming various challenges. We also wish to thank **Dr. Bibhu Prasad**, Head of the Department of Electronics and Communication Engineering, GIET University, Gunupur, for his valuable suggestions and unwavering support during the project period. The entire faculty group of the department has provided moral support and cooperation, for which we are deeply grateful. Our families have been the foundation of our mental strength, offering unwavering support and acceptance of our decision to pursue higher studies. Their encouragement and belief in us have been indispensable, and we are sincerely thankful. We also express our gratitude to the central library staff members for their kind cooperation and assistance during our research work, which has greatly contributed to the success of our project. Lastly, we thank all our friends for their timely suggestions and encouragement. Their support has been invaluable throughout this journey. We acknowledge that many more individuals have contributed to this project in various ways, and we are deeply grateful for their selfless help and support. To all who have supported us in this endeavor, we offer our sincere thanks and appreciation. Your contributions have been instrumental in the successful completion of this project.

Ponnitivalasa Devraj
Tutika Durga Prasad
Tamada Lallu Kumar
Shubham Satapathy

ABSTRACT

This project presents the design and FPGA implementation of an AHB to APB bridge using Verilog HDL on the Basys 3 FPGA board. The bridge is used to connect a high-speed AHB master interface with low-speed APB peripherals through proper protocol conversion and control logic. The design consists of an AHB master generator, AHB slave interface, APB controller finite state machine, and APB-based peripherals such as LED, timer, UART, pattern generator, and seven-segment display controller. The AHB slave interface captures address, control, and data signals from the AHB side and forwards them to the APB controller. The APB controller generates peripheral select, write enable, read/write sequencing, and handshake signals required for APB communication. The implemented system supports both write and read operations. In write mode, the bridge configures the LED, timer, and UART peripherals. In read mode, the bridge reads back peripheral data, such as the programmed timer value, and verifies correct data transfer. The FPGA output demonstrates real-time peripheral operation. The LED section displays a rotating pattern based on timer-generated ticks, the UART transmits the string `HELLO` to a serial terminal, and the seven-segment display indicates the current operation type, selected peripheral, and transfer data. Internal hardware behavior is monitored using an Integrated Logic Analyzer (ILA), which confirms correct state transitions and bus activity during read and write transactions. This project demonstrates bus protocol conversion, FSM-based controller design, peripheral interfacing, hardware debugging, and FPGA-based validation of an SoC-style communication bridge.

CONTENT

DECLARATION	i
CERTIFICATE	II
ACKNOWLEDGEMENT	III
ABSTRACT	IV
CHAPTER-1	6
<i>INTRODUCTION</i>	6
CHAPTER-2	10
<i>Literature Review</i>	10
CHAPTER-3	13
<i>System Design and Architecture</i>	13
3.1 Hardware Components:	13
3.2 Block Diagram	13
3.3 Working Principle	16
3.4 FSM Diagram:	19
CHAPTER-4	22
<i>RESULT & ANALYSIS</i>	22
4.1 Simulation Results	22
4.2 FPGA Hardware implementation and ILA Verifications	28
CHAPTER-5	33
ADVANTAGES AND APPLICATIONS	33
5.1 Advantages	33
5.2 Applications	38
CHAPTER-6	48
CONCLUSION AND FUTURE SCOPE	48
REFERENCE	49
ANNEXURE	51

CHAPTER-1

INTRODUCTION

In the modern era of digital technology, the demand for high-performance, energy-efficient, and compact electronic systems has grown significantly. From smartphones and wearable devices to industrial automation and communication systems, digital electronics play a crucial role in enabling advanced functionalities. At the heart of these systems lies the concept of integration, where multiple functional components such as processors, memory units, and peripheral devices are combined into a single platform. This integration is commonly realized through System-on-Chip (SoC) architecture, which has become the backbone of contemporary embedded system design. An SoC integrates various modules on a single chip, allowing faster communication, reduced power consumption, and improved system reliability. One of the key challenges in SoC design is ensuring efficient communication between different modules that operate at varying speeds and complexities. High-performance components such as processors and memory units require fast and reliable data transfer mechanisms, while peripheral devices such as timers, communication interfaces, and input/output modules typically operate at lower speeds and require simpler control logic. To address this challenge, standardized bus architectures are employed, which define protocols for communication between different components. Among these, the Advanced Microcontroller Bus Architecture (AMBA) has emerged as one of the most widely adopted standards in the semiconductor industry[1]. AMBA, developed by ARM, provides a family of bus protocols designed to facilitate efficient communication within SoC systems. It includes several types of buses, such as the Advanced High-performance Bus (AHB), Advanced Peripheral Bus (APB), Advanced extensible Interface (AXI), and others. Each of these buses is optimized for specific types of communication. The AHB is designed for high-speed, high-bandwidth communication and is typically used to connect processors, memory controllers, and other performance-critical components. It supports advanced features such as pipelining, burst transfers, and multiple masters, making it suitable for handling large volumes of data with minimal latency. On the other hand, the APB is a simpler bus protocol designed for low-power, low-complexity communication with peripheral devices. It does not support pipelining or burst transfers, which simplifies its design and reduces power consumption, making it ideal for interfacing with simple peripherals. The coexistence of AHB and APB within the same system necessitates a mechanism to enable communication between these two buses. Since they operate using

different protocols, timing mechanisms, and signal structures, a direct connection between them is not feasible. This is where the AHB to APB bridge plays a critical role. The bridge acts as an interface that translates transactions from the AHB protocol into APB-compatible signals. It ensures that data, address, and control signals are properly synchronized and transferred between the two domains without loss of information or timing errors. The design of such a bridge requires careful consideration of protocol differences, timing constraints, and signal integrity. The primary objective of this project is to design and implement an AHB to APB bridge using Verilog Hardware Description Language (HDL) and to validate its functionality on an FPGA platform, specifically the Basys 3 development board. Verilog HDL is widely used for modeling and designing digital systems due to its flexibility, scalability, and compatibility with synthesis tools. By using Verilog, the entire system can be described at the register-transfer level (RTL) [2], allowing precise control over hardware behavior. FPGA implementation provides a practical platform for testing and verifying the design in real time, bridging the gap between simulation and actual hardware deployment. The architecture of the proposed system is modular and consists of several interconnected components, each responsible for a specific function. The AHB master generator simulates the behavior of a processor or master device by generating read and write transactions. It produces signals such as address, control information, and data, which are sent to the AHB slave interface. The AHB slave interface acts as a receiver that captures these signals and performs necessary operations such as latching and pipelining. This ensures that the data is aligned with the timing requirements of the APB protocol before being passed to the APB controller. The APB controller is the core component of the bridge and is implemented using a finite state machine (FSM). The FSM controls the sequence of operations required for protocol conversion, including the idle state, setup phase, and enable phase. During the setup phase, the address and control signals are prepared, and the appropriate peripheral is selected using address decoding logic. In the enable phase, the actual data transfer takes place, and the required control signals such as PSEL (peripheral select), PENABLE (enable signal), and PWRITE (write control) are generated. The controller also manages handshake signals to ensure proper synchronization between the AHB and APB domains. The use of an FSM simplifies the design and ensures deterministic behavior, which is essential for reliable communication. To demonstrate the functionality of the bridge, multiple APB-based peripherals are integrated into the system. These peripherals include an LED controller, timer module, UART transmitter, pattern generator, and seven-segment display controller [3]. Each peripheral is mapped to a specific address range, allowing the APB controller to select the appropriate device based on the

incoming address. The LED controller is used to display output data visually, providing immediate feedback on system operation. The timer module generates periodic signals that can be used to control the speed of operations or introduce delays. The pattern generator creates dynamic output patterns, such as shifting or blinking LEDs, which help verify the correctness of data transfer and control logic. The UART module enables serial communication between the FPGA and an external device, such as a computer. This is particularly useful for debugging and monitoring system behavior. In this project, the UART is used to transmit the string “HELLO” to a serial terminal, demonstrating successful communication through the bridge. The seven-segment display controller adds another layer of functionality by displaying numerical or status information. It can be used to indicate the current operation mode (read or write), selected peripheral, or data being transferred, enhancing system observability. The system supports both read and write operations, making it a complete and functional communication bridge. In write mode, the AHB master sends data to configure the peripherals. For example, it can set LED patterns, initialize timer values, or trigger UART transmission. In read mode, the bridge retrieves data from the peripherals, such as reading the current timer value or status registers. This bidirectional communication capability is essential for real-world applications, where both control and monitoring operations are required. The correctness of these operations is verified through observed outputs and internal signal analysis. The implementation of the design on the Basys 3 FPGA board provides a practical demonstration of the system. FPGA platforms are widely used for prototyping and testing digital designs because they offer flexibility, reconfigurability, and real-time operation. By implementing the bridge on FPGA, the project moves beyond theoretical concepts and simulation, providing a tangible representation of system behavior [4]. The LED outputs, UART communication, and seven-segment display provide clear and immediate feedback, making it easier to verify functionality and identify potential issues. To further enhance the debugging process, the project incorporates an Integrated Logic Analyzer (ILA). The ILA is an on-chip debugging tool that allows designers to monitor internal signals in real time. It captures signals such as FSM states, address lines, control signals, and data paths, enabling detailed analysis of system behavior. This is particularly useful for identifying timing issues, verifying protocol conversion, and ensuring correct operation under different conditions. The use of ILA eliminates the need for external measurement equipment and provides a more efficient and accurate debugging process. From an educational perspective, this project provides comprehensive exposure to several important concepts in digital system design. These include bus architecture, protocol conversion, finite state machine design, peripheral interfacing,

hardware description languages, FPGA implementation, and real-time debugging techniques. By integrating these concepts into a single project, it offers a holistic understanding of how modern digital systems are designed and implemented. It also provides hands-on experience with industry-standard tools and methodologies, which is valuable for students pursuing careers in electronics and embedded systems. Moreover, the modular nature of the design allows for easy expansion and customization. Additional peripherals such as SPI, I2C, ADC, PWM, and interrupt controllers can be integrated into the APB bus to extend system functionality. Advanced features such as multiple AHB masters, arbitration mechanisms, and direct memory access (DMA) can be added to create a more complex and realistic SoC model. The inclusion of a soft-core processor, such as Micro Blaze, can further enhance the system, enabling software-controlled operation and advanced applications. The significance of this project extends beyond academic learning. The concepts and techniques demonstrated in this work are widely used in industry for designing complex digital systems. The ability to design and implement a bus bridge, integrate multiple peripherals, and validate the system on FPGA is a valuable skill set for engineers working in fields such as semiconductor design, embedded systems, and hardware development. The project also highlights the importance of debugging and verification, which are critical stages in the design process [5].

In conclusion, the AHB to APB bridge is a fundamental component in AMBA-based SoC architectures, enabling efficient communication between high-speed and low-speed modules. This project successfully demonstrates the design, implementation, and validation of such a bridge using Verilog HDL and FPGA technology. By integrating multiple peripherals and incorporating real-time debugging features, it provides a practical and comprehensive platform for understanding SoC communication architecture. The knowledge gained from this project serves as a strong foundation for future work in digital design and embedded systems, making it a valuable contribution to both academic and practical domains.

CHAPTER-2

Literature Review

“Design And Implementation of AHB To APB Bridge Using Verilog” (IJCRT, 2025)

This work presents a Verilog-based implementation of an AHB-to-APB bridge, focusing on improving design efficiency through FPGA optimization techniques. The implementation addresses clock domain challenges, particularly clock skew mitigation, and ensures synchronization between the high-speed AHB and low-speed APB interfaces through structured protocol conversion and signal alignment. The results demonstrate improved timing stability and efficient data transfer with reduced timing violations due to clock optimization; however, the lack of advanced verification methodologies such as UVM limits the validation depth and reduces confidence in reliability for complex System-on-Chip (SoC) applications.[6]

Design and Implementation of AMBA Based AHB to APB Bridge Protocol” (IJSRSET, 2024)

This paper focuses on optimizing the AHB-to-APB bridge using an enhanced finite state machine (FSM) design, particularly targeting efficient burst transfer handling. The implementation improves state transition control to reduce switching activity and enhance power efficiency in SoC environments. The results indicate improved power performance and better handling of burst transactions; however, the evaluation is limited to basic simulations without benchmarking against commercial IP cores, and the absence of detailed timing waveform analysis restricts a comprehensive understanding of real-time performance.[7]

“Design of AHB to APB Bridge” (BVM Engineering Project Report, 2020) This work presents a functional design of the AHB-to-APB bridge with a strong emphasis on verification through comprehensive testbenches. The implementation incorporates clock gating techniques on the HCLK signal to reduce dynamic power consumption, making it suitable for low-power applications, while ensuring proper protocol translation and peripheral interfacing. The results show effective functional verification and significant power reduction due to clock gating; however, the design is limited to single-master configurations and does not address multimaster scalability, which restricts its applicability in modern and complex SoC systems.[8]

“Implementation and Verification of AHB 5.0 to APB Bridge” (Harbin Engineering Journal)

This paper introduces an advanced AHB 5.0 to APB bridge design with a focus on accurate protocol translation and efficient address decoding for routing within SoC architectures. The implementation utilizes SystemVerilog-based verification environments to ensure design correctness and robustness, along with efficient integration between high-performance and peripheral subsystems. The results demonstrate reliable protocol conversion and effective

address mapping, while the use of advanced verification techniques significantly improves validation confidence, making the design suitable for high-performance and complex SoC applications.[9]. “Design and Implementation of AHB to APB Bridge using Verilog” (IJEMR, 2025) This paper proposes a modular FSM-based architecture for the AHB-to-APB bridge, emphasizing portability across FPGA and ASIC platforms with clearly defined signal flows and modular components. The implementation focuses on simulation-based validation to ensure functional correctness and design clarity. The results highlight improved flexibility and portability of the design; however, the absence of hardware implementation and lack of fault-handling mechanisms such as error detection or CRC reduce its suitability for real-world and safety-critical applications, indicating a need for enhanced reliability features [10].

Padmaprabha Jain and Satheesh Rao, “Design and Verification of Advanced Microcontroller Bus Architecture-Advanced Peripheral Bus (AMBA-APB) Protocol” (IEEE ICRACC, 2020). This paper details the design and verification of the AMBA-APB protocol using SystemVerilog and UVM, emphasizing functional coverage and assertion-based checks for peripheral interfacing. It achieves high verification coverage through constrained-random testing and protocol compliance validation; however, it lacks focus on multi-clock domain integration or performance metrics in real SoC environments, limiting scalability for heterogeneous bus systems [11]. Jaehoon Song et al., “An Efficient SOC Test Technique by Reusing On/Off-Chip Bus Bridge” (IEEE TCAS-I, Vol. 56, No. 3, 2009). The authors propose a test technique for SoCs that reuses on/off-chip bus bridges to minimize test time and hardware overhead, integrating scan chains with bridge logic for at-speed testing. Results show up to 50% reduction in test application time; yet, it overlooks advanced protocol bridges like AHB-APB and assumes uniform clock domains, reducing applicability to modern multi-frequency AMBA designs [12]. Sangik Choi and Shinwook Kang, “Implementation of an On-Chip Bus Bridge between Heterogeneous Buses with Different Clock Frequencies” (IEEE IDEAS, 2005).

This work implements a bridge for heterogeneous buses operating at varying clock frequencies, using asynchronous FIFOs and handshaking for data synchronization in mobile SoCs. It demonstrates robust cross-domain transfers with low latency; however, the absence of formal verification or AMBA-specific compliance testing raises concerns about reliability in standardized protocols like APB or AHB [13]. D. Flynn, “AMBA: Enabling Reusable On-Chip Designs” (IEEE Micro, Jul/Aug 1997). Flynn introduces the AMBA on-chip bus standard, highlighting its modular AHB, APB, and ASB components for reusable IP cores in ARM-based systems. It paved the way for efficient SoC integration with pipelined high-performance buses; still, as an early overview, it predates modern verification tools and clock skew challenges,

offering limited practical implementation guidance [14]. Wang Zhonghai et al., “Designing AHB/PCI Bridge” (Proc. 4th Int. Conf. on ASIC, 2001).The paper designs an AHB-to-PCI bridge for high-speed peripheral connectivity, employing burst transfers and address mapping to handle protocol mismatches. Simulation results confirm efficient bandwidth utilization; however, it ignores low-power APB extensions and lacks FPGA prototyping, constraining its relevance to contemporary low-speed peripheral bridging [15]. A. Kharade and Jayashree V., “VLSI Design of AMBA Based AHB2APB Bridge” (IEEE ICECS, 2021).This study presents a VLSI implementation of an AHB-to-APB bridge in Verilog, focusing on address decoding, handshaking, and error handling for SoC peripherals. Post-synthesis results show area-efficient design with timing closure; yet, it does not incorporate clock skew minimization or advanced UVM verification, potentially impacting multi-clock reliability [16]. M. K. Kumar and Amritha Sajja, “Design and FPGA Implementation of AMBA APB Bridge with Clock Skew Minimization Technique” (IEEE VDAT, 2021).The authors implement an AMBA APB bridge on FPGA with a novel clock skew minimization using dynamic phase alignment and gray coding for synchronizers. FPGA results indicate 20% timing improvement and reduced metastability; however, the design omits AHB integration and comprehensive testbenches, limiting its use in full bus hierarchies [17]. Jain and Rao present a UVM-based design and verification environment for the AMBA-APB protocol, ensuring strict compliance with the APB specification using constrained-random testing and functional coverage. The work models the APB finite-state machine and validates basic read/write transactions through a layered UVM testbench. Results show error-free operation and full coverage of key protocol states, confirming correctness for low-speed peripherals. However, the study focuses on a single synchronous clock domain and does not address multi-clock issues such as clock skew or asynchronous bridging from AHB/AXI. It also lacks performance and power analysis, limiting its use in advanced SoC integration [18].

CHAPTER-3

System Design and Architecture

3.1 Hardware Components:

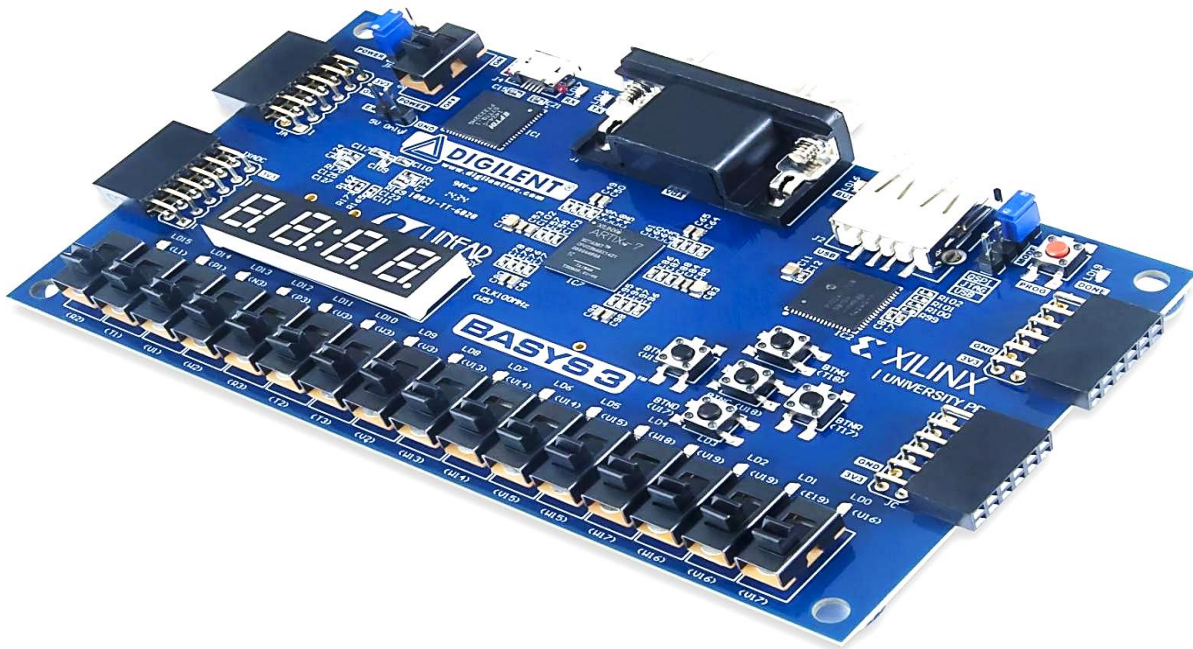


Fig.1.1.1 Basys 3 FPGA board

The Basys 3 FPGA Board serves as the primary hardware platform for the implementation and validation of the AHB-to-APB bridge project, transitioning the design from a theoretical Verilog HDL simulation into a physical System-on-Chip (SoC) environment. At its core is the Xilinx Artix-7 FPGA (XC7A35T-1CPG236C), which provides an industrial-grade architecture featuring 33,280 logic cells, 5,200 slices, and 1,800 Kbits of Block RAM. These resources are essential for managing the high-speed 100 MHz system clock (\$Hclk\$) and the complex multi-state logic of the AMBA protocol. By using the board's dedicated clock management tiles and high-performance fabric, the modular components of the bridge—including the AHB slave interface and the 9-state FSM APB controller—are synthesized into reliable hardware gates.

Beyond the core silicon, the Basys 3 provides integrated physical interfaces that allow for immediate system verification. The board's 16 User LEDs are mapped to the APB pattern generator, providing a visual confirmation of data decoding, while the four-digit seven-segment

display is utilized to output real-time 8-bit data values. Additionally, the 16 slide switches and 5 push buttons act as manual triggers for system resets (\$Hresetn\$) and bus transactions. This hardware-in-the-loop approach ensures that the design functions correctly under real electrical conditions, utilizing the board's 3.3V LVCMOS I/O standards to maintain signal integrity between the FPGA and its peripherals.

A sophisticated aspect of using the Basys 3 is the integration of the Integrated Logic Analyzer (ILA) via the onboard micro-USB JTAG interface. During the implementation phase, ILA debug cores are embedded directly into the FPGA fabric, utilizing the Artix-7's internal memory to capture real-time signals. This is particularly crucial for verifying the 9-state FSM transitions within the APB controller and the precise timing of the 9600 Baud rate generator in the UART module. By capturing these waveforms directly from the chip, any discrepancies between the intended protocol behavior and the physical implementation can be diagnosed with nanosecond precision, which is impossible to achieve through external observation alone.

Finally, the use of the Basys 3 enhances the industry relevance of the project by following a standard professional FPGA design flow. This includes the process of synthesis, placement, routing, and bitstream generation within the Vivado Design Suite, specifically utilizing the Basys 3 "Board Files" to automate pin constraints. The result is a validated hardware prototype that proves the efficiency of the AHB-to-APB bridge in a constrained resource environment. By successfully deploying the design onto this platform, the project demonstrates how high-performance bus architectures are effectively throttled for lower-power, slower peripherals in modern, energy-efficient computing systems.

3.2 Block Diagram

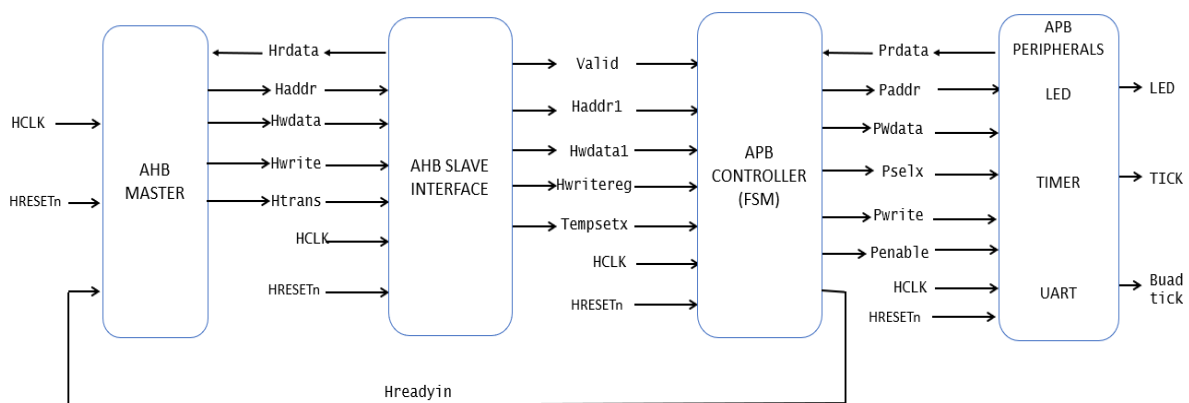


Fig.3.1 block diagram of AHB to APB Bridge

The heart of the AHB-to-APB bridge is the APB Controller, which is implemented as a sequential 9-state Finite State Machine (FSM). The primary objective of this FSM is to manage the temporal and protocol-level disparities between the high-performance **Advanced High-performance Bus (AHB)** and the power-efficient **Advanced Peripheral Bus (APB)**. Because the AHB protocol operates with a pipelined address and data phase—meaning the address of a transaction is provided one clock cycle before the data—it cannot be directly connected to APB peripherals, which require the address, data, and control signals to be perfectly synchronized and stable. This FSM acts as a "protocol translator," stalling the AHB master when necessary and ensuring that the APB peripherals receive clean, stable signals.

The FSM begins its operation in the **ST_RESET** state. Upon the assertion of the system's active-low reset signal ($\$Hresetn = 0\$$), the controller initializes all internal registers and de-asserts all APB control signals, such as PSELx, PENABLE, and PWRITE. This ensures that no peripheral is accidentally triggered during power-up or system reset. Once the reset signal is released, the FSM transitions to the **ST_IDLE** state. In the Idle state, the controller continuously monitors the Valid signal from the AHB Slave Interface. As long as Valid remains at logic 0, the FSM stays in Idle, consuming minimal power and keeping the APB bus inactive. The transition from Idle depends entirely on the nature of the incoming AHB transaction. If the AHB Slave Interface detects a valid address within the peripheral memory map ($\$Valid=1\$$) and the master indicates a read operation ($\$Hwrite=0\$$), the FSM moves into the **ST_READ** state. In this state, the controller asserts the PSELx signal for the specific peripheral (LED, Timer, or UART) and prepares the address for the APB bus. In the following clock cycle, the FSM enters **ST_REENABLE**, the "Access" phase of the APB protocol. Here, the PENABLE signal is driven high, signaling the peripheral to put its data onto the bus. Once the data is sampled, the FSM either returns to Idle or, if another read is immediately pending, cycles back to the Read state to maintain throughput.

The write logic of the FSM is more sophisticated due to the pipelined nature of the AHB bus. When a write transaction is detected ($\$Valid=1\$$ and $\$Hwrite=1\$$), the FSM first moves to the **ST_WWAIT** state. This is a critical "wait" state where the controller captures the address from the previous cycle and waits for the AHB Master to provide the corresponding data ($\$Hwdata\$$). Because the AHB master is much faster, the FSM pulls the Hreadyout signal to logic 0 during this time, effectively forcing the AHB Master into a "wait state" so it does not send the next address until the bridge is ready. This synchronization is what prevents data loss during high-speed transfers.

Following the wait state, the FSM enters the **ST_WRITE** or **ST_WRITE_P** states. These represent the "Setup" phase of an APB write transaction. In these states, the bridge drives the PSELx, PWRITE, and the buffered Pwdata onto the APB bus. The transition then moves to the **ST_WENABLE** or **ST_WENABLE_P** states, which act as the "Access" phase. During these Enable states, the PENABLE signal is asserted, and the peripheral physically captures the data. The "P" (Pipelined) versions of these states—**ST_WRITE_P** and **ST_WENABLE_P**—are specialized logic paths that allow the bridge to handle back-to-back write transactions. Instead of returning to Idle after every write, the FSM can jump directly between these pipelined states, significantly increasing the bus efficiency when the AHB Master is performing burst writes to the UART or LED registers.

Ultimately, the complexity of this 9-state FSM is a design requirement for **AMBA 3.0 compliance**. By meticulously managing the handshaking signals—specifically Valid, Hwrite, and the feedback signal Hreadyout—the controller ensures that the system is robust against timing violations. In a professional SoC design, such an FSM is the standard method for protecting slow, low-power peripherals from the high-frequency demands of the main processor. For the Basys 3 implementation, this FSM ensures that every bit sent by the AHB generator is accurately captured by the UART or reflected on the LEDs, regardless of the 100 MHz frequency of the system clock. This makes the project not only a functional success but an industry-relevant example of synchronous digital design and bus interfacing.

3.3 Working Principle

System Architecture and Master Initiation Logic

The fundamental architectural objective of this project is the design and hardware implementation of an AHB-to-APB Bridge on the Xilinx Artix-7 FPGA, specifically utilizing the Digilent Basys 3 development platform. In modern System-on-Chip (SoC) design, a high-speed system bus like the Advanced High-performance Bus (AHB) is necessary for high-bandwidth components, while the Advanced Peripheral Bus (APB) is optimized for low-power, simpler peripherals. This project bridges that gap.

Upon the de-assertion of the global system reset ($\overline{\text{Hresetn}}$), the AHB Master logic is hardcoded to initiate a deterministic sequence of transactions. This "Master" simulates an ARM-based processor by generating a burst of write operations targeted at the memory-mapped addresses of the LED, Timer, and UART modules. This initiation sequence is critical for setting the initial configuration of the hardware registers, such as defining the timer's delay

constant or sending the first string of characters to the UART buffer. By utilizing Verilog HDL, the master ensures that these transactions adhere to strict timing requirements, setting the stage for the bridge to perform its translation duties.

AHB Slave Interface and Multi-Level Address Decoding

The AHB Slave Interface serves as the primary gateway between the high-speed bus and the bridge controller. It is designed to capture the AHB Address Phase signals, including Haddr (32-bit address), Hwrite (direction), Htrans (transfer type), and Hsize (data width). Because AHB is a pipelined protocol, the Slave Interface must accurately latch these control signals and hold them until the corresponding Data Phase begins.

The interface includes a specialized Address Decoder that parses the incoming 32-bit address to generate the tempselx signal. This signal is a 3-bit "pre-select" vector where each bit corresponds to a specific APB peripheral: 001 for the LED Controller, 010 for the System Timer, and 100 for the UART Module. This localized decoding is essential for modularity; it allows the system to support multiple peripherals sharing the same APB bus without signal collisions. Furthermore, the interface generates a Valid signal, which notifies the APB Controller that a legitimate transaction is pending and that the FSM should break from its IDLE state.

APB Controller and the 9-State FSM Protocol Logic

The APB Controller functions as the "brain" of the bridge, housing a sophisticated 9-state Finite State Machine (FSM). The primary challenge this FSM solves is the synchronization of the AHB pipelined data phase with the non-pipelined, two-cycle APB protocol. For a Write Operation, the FSM transitions through a sequence of WAIT, WRITE, and WENABLE states.

During the WAIT state, the controller stalls the AHB master by pulling Hreadyout to logic 0, allowing time for the write data (Hwdata) to arrive from the master. In the WRITE (Setup) phase, the controller drives the Pselx and Paddr signals, followed by the WENABLE (Access) phase, where Penable is driven high to strobe the data into the peripheral register. This meticulous state-stepping ensures that the peripheral receives stable data that meets the setup and hold time requirements of the APB standard, preventing data corruption that would otherwise occur at high clock frequencies.

Advanced Peripheral Interaction: Timer and Pattern Synchronization

The interaction between the bridge and the peripherals demonstrates the practical application of bus-driven control. The System Timer Peripheral acts as a frequency divider, taking the 100MHz system clock and counting down from a value written during the AHB write phase. Once the counter reaches zero, it generates a single-cycle pulse known as a tick.

This tick is fed into a Pattern Module, which governs the 16 LEDs on the Basys 3 board. Rather than simply displaying a static value, the module uses the timer's frequency to create a dynamic blinking effect. The 4-bit data written by the Master acts as a "mask." For example, if the value 1011 is written, the Pattern Module ensures that LED 3, LED 1, and LED 0 blink in perfect synchronization with the timer's heartbeat, while LED 2 remains dark. This demonstrates that the bridge is not just moving data, but is enabling complex, synchronized hardware behaviors across different logic domains.

UART Serial Communication and Host-Side Monitoring

One of the most complex peripherals in this project is the UART (Universal Asynchronous Receiver-Transmitter). When the AHB Master writes a byte to the UART's address space, the bridge transfers that data to the UART's internal transmit buffer. The UART then utilizes a local Baud Rate Generator to create a baud_tick (typically for 9600 Baud).

Every time the baud tick pulses, the UART shifts one bit of the data out onto the physical tx line of the FPGA. To confirm that the bridge successfully transported the data from the high-speed bus to this serial stream, the Basys 3 is connected to a PC via its USB-UART bridge. By using the PuTTY terminal application, the user can see the actual characters sent by the AHB Master. This end-to-end verification proves that the bridge is capable of managing time-sensitive communication protocols where precise bit-timing is mandatory for data integrity.

Bi-Directional Handshaking and Read Operation Logic

While write operations push data "down" to peripherals, the Read Operation validates the "upward" path of the bridge. After the master completes its configuration writes, it issues a read command to the Timer Peripheral (Address 0x8400_0000) to confirm the configuration was successful. During this phase, the FSM enters the READ and RENABLE states.

The Pwrite signal is held at logic 0, signaling a data fetch. The Timer Peripheral responds by placing its internal 32-bit register value onto the Prdata bus. The bridge then captures this value and routes it back to the AHB Master via the Hrdata bus. This bi-directional handshaking is the

hallmark of a robust bus architecture; it allows the master to perform "Read-After-Write" checks, ensuring that the system is functioning correctly and that no "phantom" writes occurred due to timing violations.

Hardware Visualization, Debugging, and ILA Synthesis

To make the internal workings of the bridge visible to the designer, the project incorporates two layers of hardware feedback. On the physical board, the Seven-Segment Display serves as a real-time status dashboard. The first digit identifies the transaction type (R/W), the second digit shows which of the three peripherals is active, and the final two digits display the hexadecimal value of the data being moved.

For professional-grade debugging, the Integrated Logic Analyzer (ILA) is synthesized directly into the Artix-7 fabric. By attaching "probes" to internal signals like fsm_state, Penable, and valid, the designer can use Vivado's Hardware Manager to view digital waveforms as they occur on the actual silicon. This in-system debugging confirms that the 9-state FSM is transitioning correctly and that the bridge is operating at the full 100MHz clock speed without setup or hold time failures.

3.4 FSM Diagram:

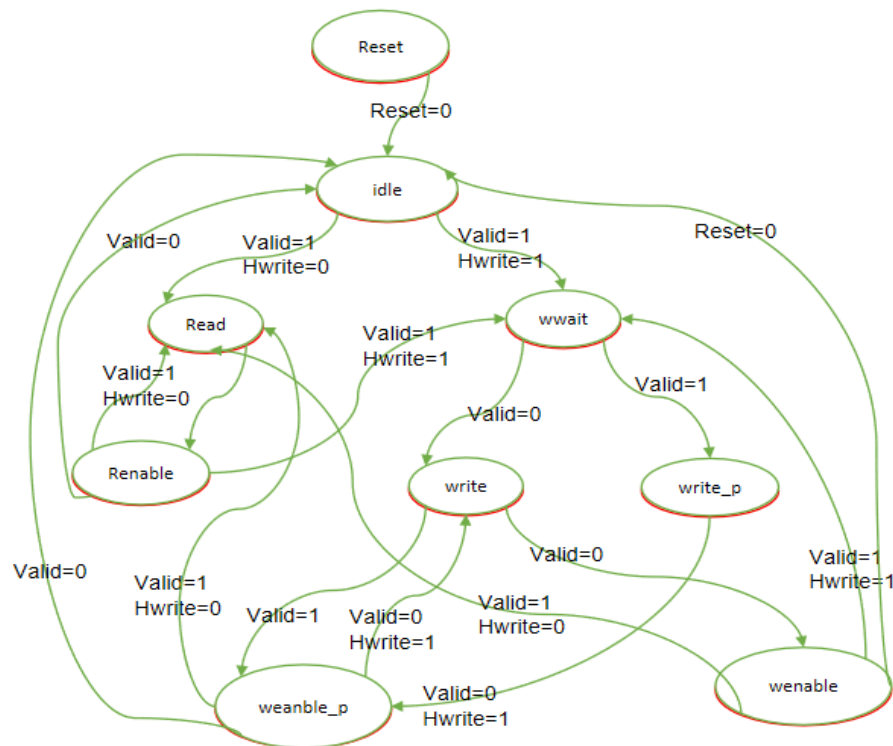


Fig.3.2 FSM diagram of AHB to APB Bridge

The heart of the AHB-to-APB bridge is the APB Controller, which is implemented as a

sequential 9-state Finite State Machine (FSM). The primary objective of this FSM is to manage the temporal and protocol-level disparities between the high-performance **Advanced High-performance Bus (AHB)** and the power-efficient **Advanced Peripheral Bus (APB)**. Because the AHB protocol operates with a pipelined address and data phase—meaning the address of a transaction is provided one clock cycle before the data—it cannot be directly connected to APB peripherals, which require the address, data, and control signals to be perfectly synchronized and stable. This FSM acts as a "protocol translator," stalling the AHB master when necessary and ensuring that the APB peripherals receive clean, stable signals.

The FSM begins its operation in the **ST_RESET** state. Upon the assertion of the system's active-low reset signal ($\$Hresetn = 0\$$), the controller initializes all internal registers and de-asserts all APB control signals, such as **PSELx**, **PENABLE**, and **PWRITE**. This ensures that no peripheral is accidentally triggered during power-up or system reset. Once the reset signal is released, the FSM transitions to the **ST_IDLE** state. In the Idle state, the controller continuously monitors the Valid signal from the AHB Slave Interface. As long as Valid remains at logic 0, the FSM stays in Idle, consuming minimal power and keeping the APB bus inactive. The transition from Idle depends entirely on the nature of the incoming AHB transaction. If the AHB Slave Interface detects a valid address within the peripheral memory map ($\$Valid=1\$$) and the master indicates a read operation ($\$Hwrite=0\$$), the FSM moves into the **ST_READ** state. In this state, the controller asserts the **PSELx** signal for the specific peripheral (LED, Timer, or UART) and prepares the address for the APB bus. In the following clock cycle, the FSM enters **ST_REENABLE**, the "Access" phase of the APB protocol. Here, the **PENABLE** signal is driven high, signaling the peripheral to put its data onto the bus. Once the data is sampled, the FSM either returns to Idle or, if another read is immediately pending, cycles back to the Read state to maintain throughput.

The write logic of the FSM is more sophisticated due to the pipelined nature of the AHB bus. When a write transaction is detected ($\$Valid=1\$$ and $\$Hwrite=1\$$), the FSM first moves to the **ST_WWAIT** state. This is a critical "wait" state where the controller captures the address from the previous cycle and waits for the AHB Master to provide the corresponding data ($\$Hwdata\$$). Because the AHB master is much faster, the FSM pulls the **Hreadyout** signal to logic 0 during this time, effectively forcing the AHB Master into a "wait state" so it does not send the next address until the bridge is ready. This synchronization is what prevents data loss during high-speed transfers.

Following the wait state, the FSM enters the **ST_WRITE** or **ST_WRITE_P** states. These represent the "Setup" phase of an APB write transaction. In these states, the bridge drives the PSELx, PWRITE, and the buffered Pwdata onto the APB bus. The transition then moves to the **ST_WENABLE** or **ST_WENABLE_P** states, which act as the "Access" phase. During these Enable states, the PENABLE signal is asserted, and the peripheral physically captures the data. The "P" (Pipelined) versions of these states—**ST_WRITE_P** and **ST_WENABLE_P**—are specialized logic paths that allow the bridge to handle back-to-back write transactions. Instead of returning to Idle after every write, the FSM can jump directly between these pipelined states, significantly increasing the bus efficiency when the AHB Master is performing burst writes to the UART or LED registers.

Ultimately, the complexity of this 9-state FSM is a design requirement for **AMBA 3.0 compliance**. By meticulously managing the handshaking signals—specifically Valid, Hwrite, and the feedback signal Hreadyout—the controller ensures that the system is robust against timing violations. In a professional SoC design, such an FSM is the standard method for protecting slow, low-power peripherals from the high-frequency demands of the main processor. For the Basys 3 implementation, this FSM ensures that every bit sent by the AHB generator is accurately captured by the UART or reflected on the LEDs, regardless of the 100 MHz frequency of the system clock. This makes the project not only a functional success but an industry-relevant example of synchronous digital design and bus interfacing.

CHAPTER-4

RESULT & ANALYSIS

4.1 Simulation Results

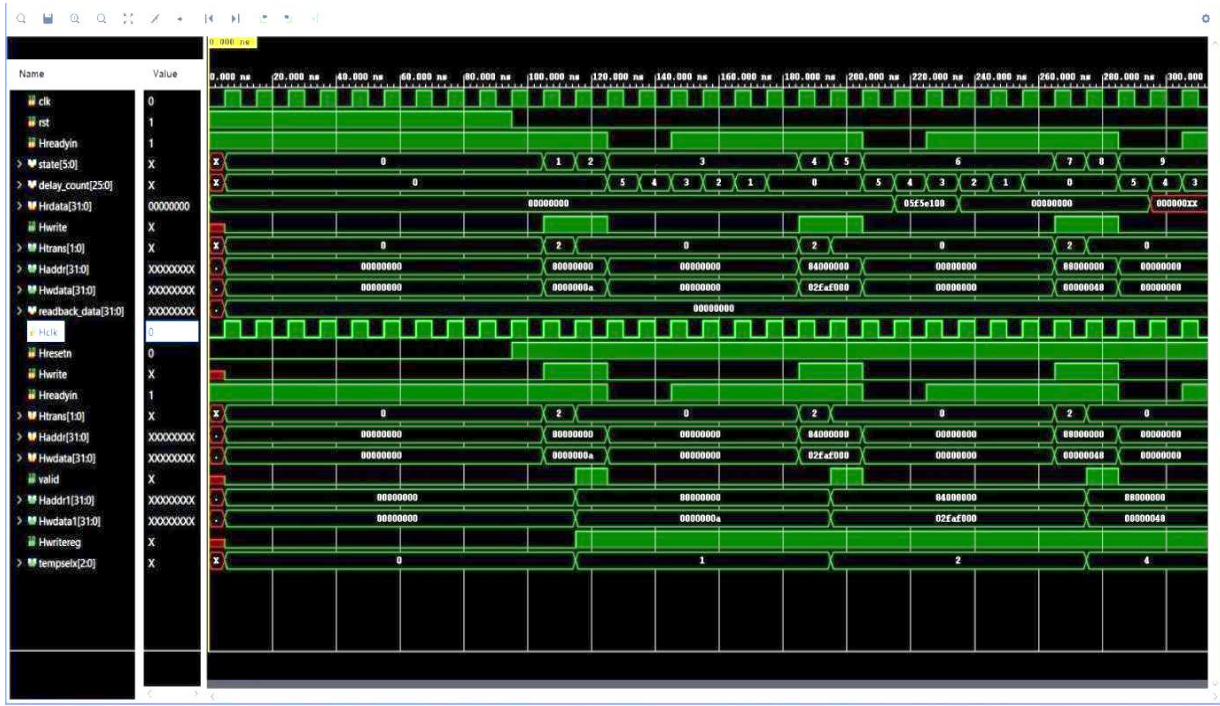


Fig 4.1.1 AHB master and AHB slave simulation waveform 1

The waveform captures the **AHB Master's autonomous generation** of a timed write sequence, representing the critical initialization phase of the SoC. Upon the de-assertion of the system reset (\$Hresetn\$), the master drives primary bus signals including Haddr, Hwdata, Hwrite, and Htrans to define the transaction parameters. Because the AHB protocol utilizes a **pipelined architecture**, the address phase and data phase overlap, requiring the bridge to be perfectly synchronized with the **100 MHz** system clock to ensure high-speed throughput without data loss. The **AHB Slave Interface** acts as the immediate receiver, capturing these high-speed bursts into local buffers identified as Haddr1, Hwdata1, and Hwritereg. This internal registration ensures the bridge maintains a stable copy of the master's request even after the main AHB bus moves to a new transaction, allowing the slower APB logic to process the data without missing high-speed transitions.

Key Waveform Observations:

- **LED Selection (\$tempsele = 1\$):** Targets the LED peripheral at address **0x8000_0000** with a captured data value of **0x0000_000A**.
- **Timer Selection (\$tempsele = 2\$):** Targets the System Timer at address **0x8400_0000**

with the write value **0x02FA_F000** to set the "tick" frequency.

- **UART Selection (\$stmpselx = 4\$):** Targets the UART module at address **0x8800_0000**, delivering sequential ASCII data bytes for serial transmission.
- **Address Integrity:** The continuous progression of the Haddr signal confirms the successful mapping and isolation of the AHB memory space for each peripheral.

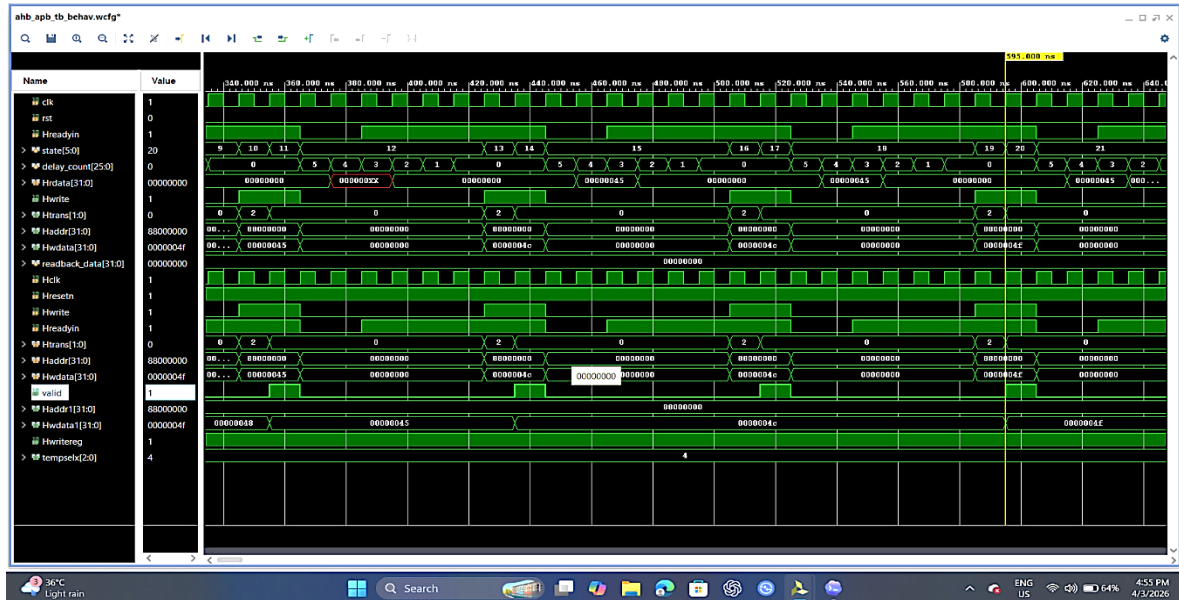


Fig 4.1.2 AHB master and AHB slave simulation waveform 2

The waveform illustrates a high-speed, consecutive **UART write sequence** occurring immediately after the system's LED and Timer configurations are finalized. This phase is characterized by the **AHB Master** repeatedly targeting the UART's memory-mapped address of **0x8800_0000**, utilizing the bridge to serialize a string of data. The **AHB Slave Interface** demonstrates its ability to handle back-to-back transactions by capturing the data phase of each character while the address phase for the subsequent character is already in progress.

The integrity of the bridge is confirmed by the behavior of the valid signal, which pulses in synchronization with each successful write transfer. This indicates that the **APB Controller FSM** is successfully transitioning between its setup and access states to clear the AHB buffer for the next incoming byte. This sequence proves that the bridge is capable of maintaining a continuous data stream, ensuring that the UART peripheral receives the intended message without timing violations or data overrun.

Key Waveform Observations:

- **UART Data Payload:** The Hwdata1 signal captures a sequence of hex values: **0x48, 0x45, 0x4C, 0x4C, 0x4F**.

- **Character Encoding:** These values correspond to the ASCII characters "H", "E", "L", "L", "O", intended for transmission to the PC terminal.
- **Peripheral Mapping:** The signal tempselx = 4 remains active throughout the burst, confirming the persistent selection of the UART peripheral.
- **Transaction Validation:** Periodic valid pulses confirm that each character has been successfully accepted by the bridge and passed to the APB domain.

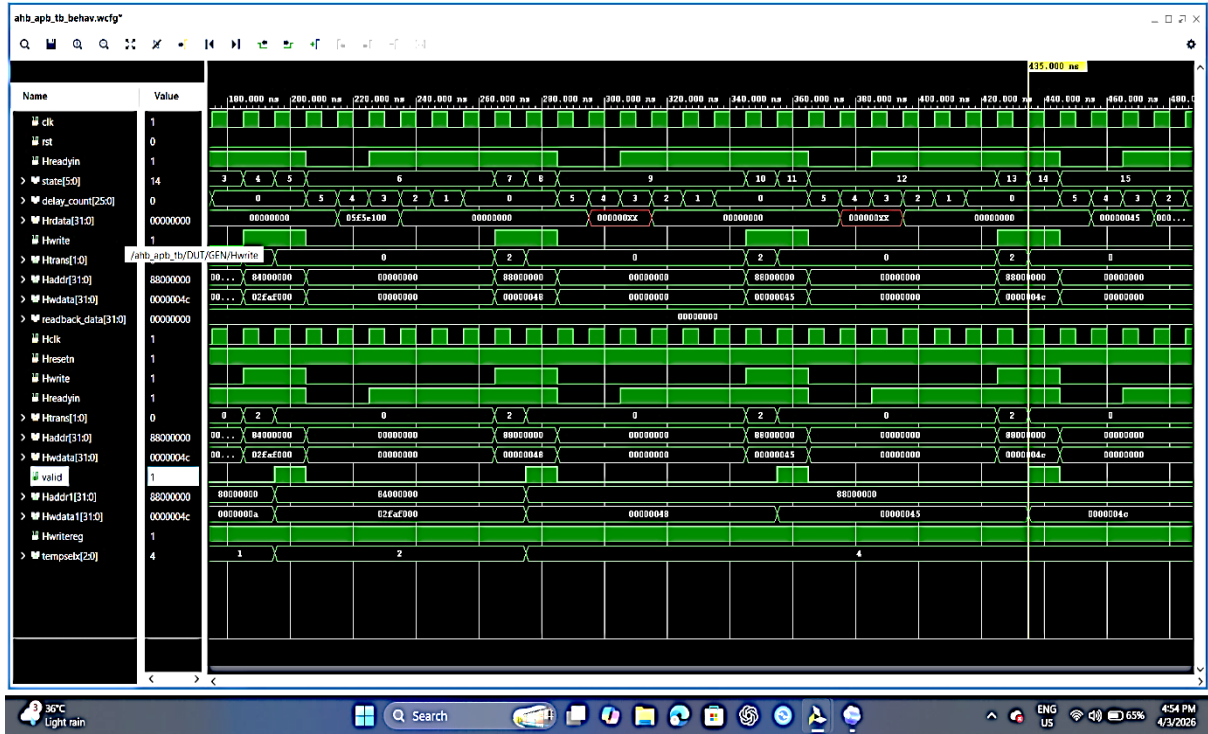


Fig 4.1.3 AHB master and AHB slave simulation waveform 3

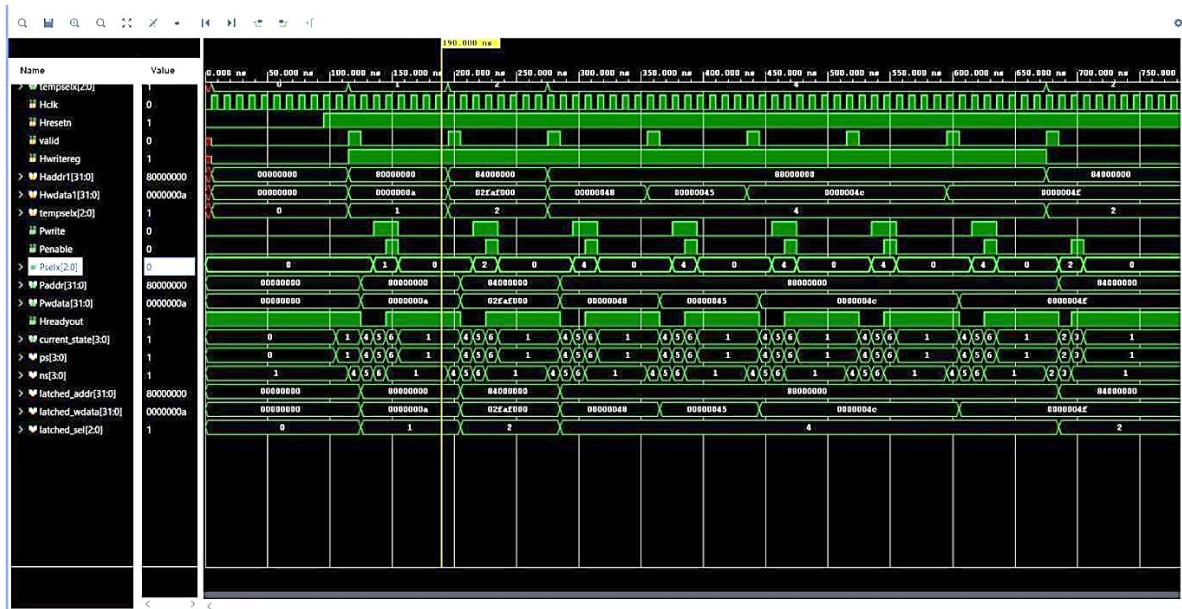
The waveform provides a comprehensive visualization of the **AHB-side write activity**, detailing the sequential initialization of the system's primary peripherals. Following the protocol's pipelined nature, the **AHB Master** successfully coordinates address and data phases to configure the LED, Timer, and UART modules. The **AHB Slave Interface** demonstrates high-integrity handshaking by accurately latching each transaction into internal buffers, ensuring that selection signals remain stable for the subsequent APB transfer. This orderly address progression validates the decoding logic, proving the bridge can manage back-to-back high-speed transactions without bus contention.

Key Waveform Observations:

- **LED Configuration:** The master issues a write of **0x0000000A** to establish the initial blinking mask.
- **Timer Initialization:** A data value of **0x02FAF000** is written to define the precise system

"tick" frequency.

- **UART Data Stream:** The master transmits ASCII bytes **0x48 ('H')**, **0x45 ('E')**, and **0x4C ('L')** for serial transmission.
- **Address Integrity:** Continuous Haddr progression confirms correct memory-mapped peripheral isolation in the intended sequence.



Fig

Fig:4.1.4 APB controller simulation waveform

The waveform provides a comprehensive visualization of the **AHB-side write activity**, detailing the sequential initialization of the system's primary peripherals. Following the protocol's pipelined nature, the **AHB Master** successfully coordinates address and data phases to configure the LED, Timer, and UART modules. The **AHB Slave Interface** demonstrates high-integrity handshaking by accurately latching each transaction into internal buffers, ensuring that selection signals remain stable for the subsequent APB transfer. This orderly address progression validates the decoding logic, proving the bridge can manage back-to-back high-speed transactions without bus contention.

Key Waveform Observations:

- **LED Configuration:** Master writes **0x0000000A** to establish the initial blinking mask.
- **Timer Initialization:** Data value **0x02FAF000** is written to define the precise system "tick" frequency.
- **UART Data Stream:** Master transmits ASCII bytes—**0x48 ('H')**, **0x45 ('E')**, and **0x4C ('L')**—for serial transmission.

- **Address Integrity:** Continuous Haddr progression confirms correct memory-mapped peripheral isolation in the intended sequence.

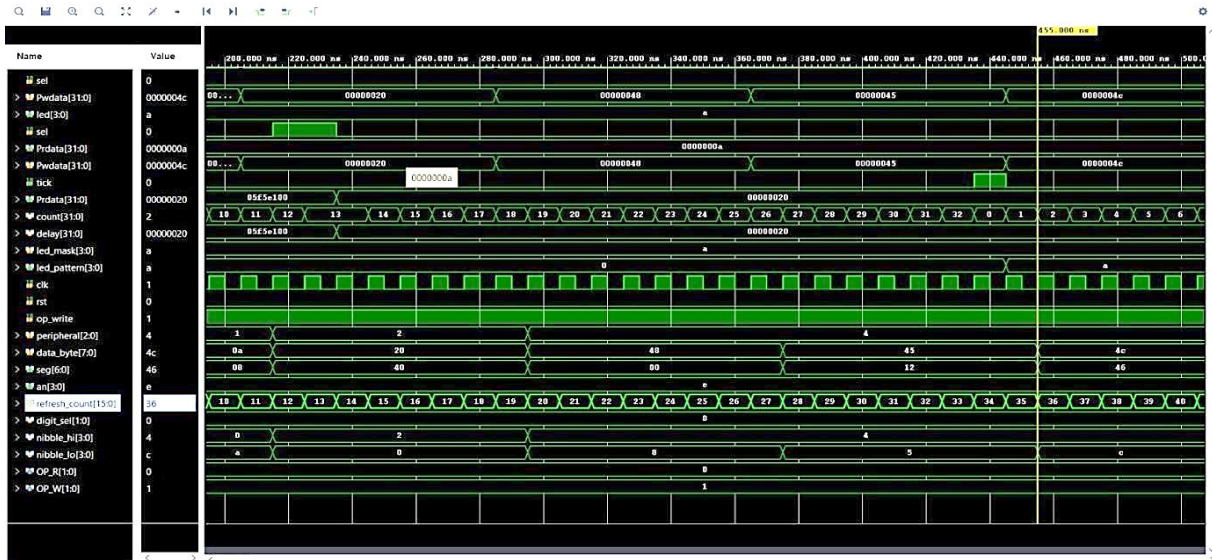


Fig 4.1.5 Peripheral simulation waveform 1

The waveform demonstrates the **real-time status-display logic** of the bridge, which translates internal protocol signals into a human-readable format on the Basys 3 hardware. The peripheral[2:0] signal serves as a one-hot indicator, transitioning through values of **1, 2, and 4** to identify active access to the LED, Timer, and UART modules, respectively. Throughout these phases, the op_write signal is held at logic high, providing constant verification that the **AHB Master** is performing write transactions. Simultaneously, the data_byte signal dynamically updates to reflect the internal Pwdata values, ensuring that the exact data being sent to the peripheral is also being routed to the display controller.

Key Status-Display Observations:

- **Peripheral Identification:** The peripheral signal transitions confirm that the **Address Decoder** is successfully isolating each module for the display logic.
- **Operation Confirmation:** The persistence of op_write confirms that the **APB Controller** is in a write state, which is reflected as a "W" on the physical FPGA display.
- **Data Serialization:** Signals such as nibble_hi and nibble_lo indicate how the 8-bit data_byte is split into two 4-bit hex characters for the seven-segment array.
- **Display Multiplexing:** The digit_sel and an (anode) signals show the high-speed switching between the four display digits, allowing the system to show the operation type, peripheral number, and data byte simultaneously without flicker.

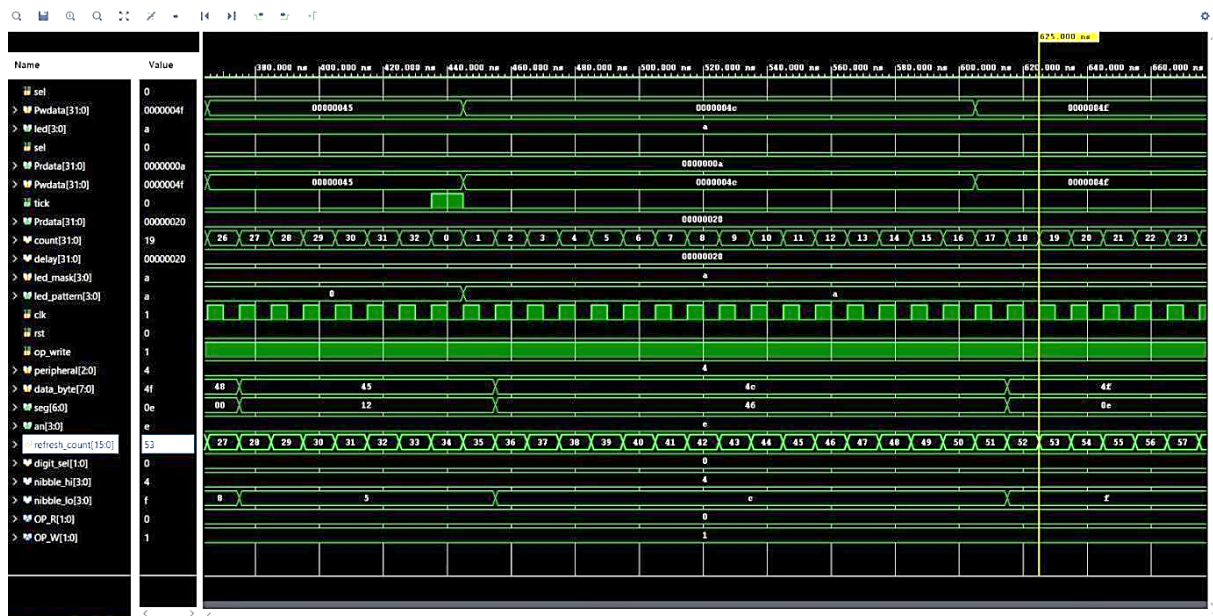


Fig 4.1.6 Peripheral simulation waveform 2

The waveform illustrates the concluding phase of the **UART write sequence** and its immediate synchronization with the hardware-level status display logic. As the AHB Master completes the string transmission, the system captures the final ASCII data bytes—**0x45 (E)**, **0x4C (L)**, and **0x4F (O)**—and routes them through the APB bridge to the UART peripheral. Simultaneously, the display controller intercepts these values, updating the `data_byte` register to ensure the seven-segment display provides a real-time visual trace of the bus activity. The internal signals `nibble_hi` and `nibble_lo` demonstrate the hexadecimal decoding process, splitting each 8-bit character into two 4-bit values to drive the physical display segments. This synchronized behavior confirms that the **APB Controller FSM** and the display multiplexing logic are working in tandem, providing a high-integrity monitoring interface that matches the serial data observed on the PC terminal.



Fig 4.1.7 Peripheral simulation waveform 3

The waveform captures the **LED peripheral's** transition from an AHB-driven command to a physical hardware output. Upon receiving the target address **0x8000_0000** and the associated write data **0x0000_000A**, the bridge successfully captures and registers this value as the internal `led_mask`. This mask acts as the primary control signal for the **Pattern Block**, which determines the active state of each individual LED.

Simultaneously, the waveform illustrates the integration of the **Timer Peripheral's** delay value, confirming that the blinking frequency is governed by the programmed timer interval. By combining the bitwise `led_mask` with the periodic **timer tick**, the system generates a synchronized `led_pattern` that pulses only the designated LEDs. This visual verification proves that the bridge is effectively coordinating multi-domain signals—address, data, and timing—to achieve precise real-time hardware control on the **Basys 3** platform.

4.2 FPGA Hardware implementation and ILA Verifications

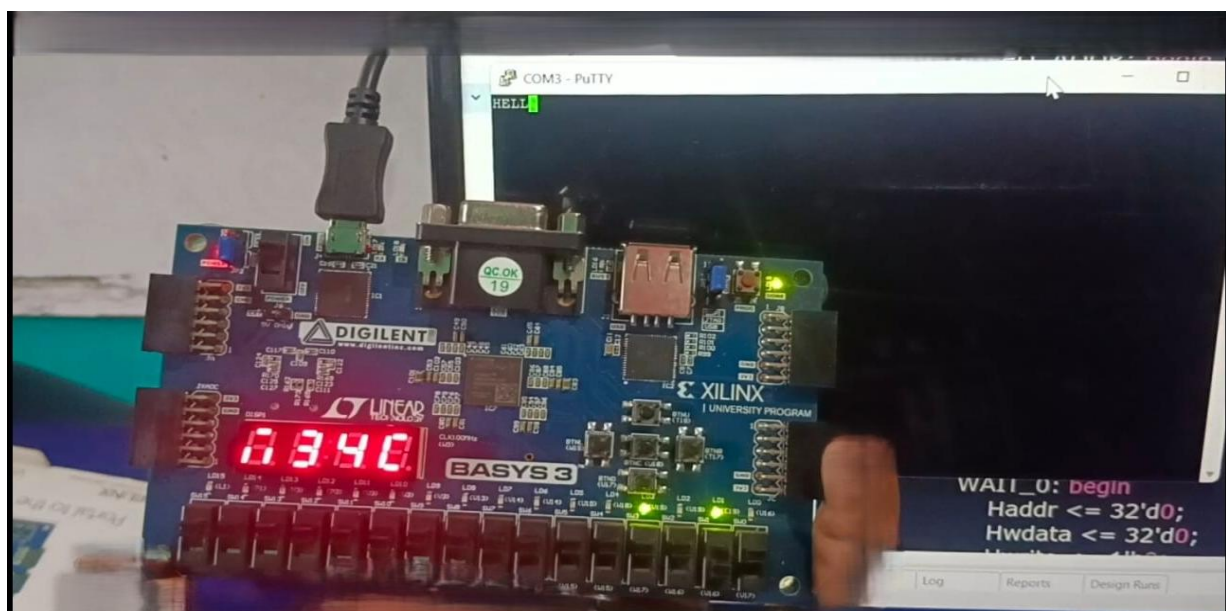


Fig 4.1.8 7-Segment Display Status During UART Write Operation

The **Basys 3** FPGA's **7-segment display** provides a critical real-time hardware status of the **AHB-to-APB bridge** during active data transfers. A "W" on the first digit confirms an active **WRITE operation** (`Pwrite=1`), while the second digit displays "3," representing the UART peripheral selection derived from the **Address Decoder** (`Pselx=100`). The final two digits capture the hex value **0x4C**, which is the specific data byte currently being routed through `Pwdata` for serial transmission. This visual feedback confirms that the **APB Controller FSM** has successfully

transitioned into the **WENABLE (Access) state**, driving Penable high to finalize the transaction. By mirroring the internal ILA signals on the physical display, the system provides an immediate verification that the AHB Master's commands are being correctly translated and delivered to the UART module for external observation on **PuTTY**.

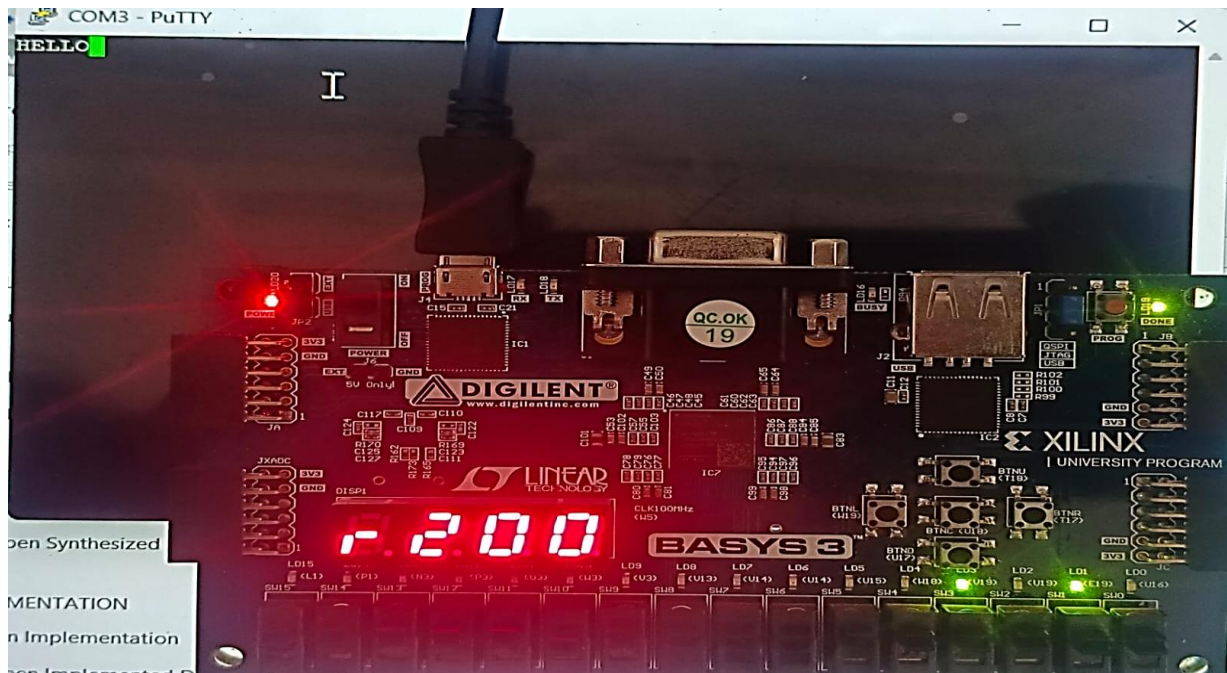


Fig 4.1.9 FPGA Hardware Output

The figure shows the successful hardware implementation of the AHB to APB bridge on the Basys 3 FPGA board. After reset, the AHB master generates a sequence of APB write transactions followed by a read transaction. First, a write is performed to the LED peripheral using address 0x80000000, where the LED control data is stored. Next, a write is performed to the Timer peripheral using address 0x84000000, where the timer delay value is programmed. After that, consecutive write transactions are performed to the UART peripheral using address 0x88000000, with data bytes corresponding to ASCII values of the characters H, E, L, L, and O.

These UART write transactions are verified on the PC using PuTTY, where the string HELLO is displayed. This confirms that the UART peripheral correctly receives APB write data and transmits it serially through the FPGA UART interface.

After completion of the write operations, the bridge performs a read operation from the Timer peripheral. The seven-segment display shows the final transaction status as r200, where r indicates a read operation, 2 indicates Timer peripheral selection, and 00 represents the lower byte of the returned read data. Thus, the hardware result confirms successful execution of

LED write, Timer writes, UART write sequence, UART output verification through PuTTY, and final timer readback on FPGA.

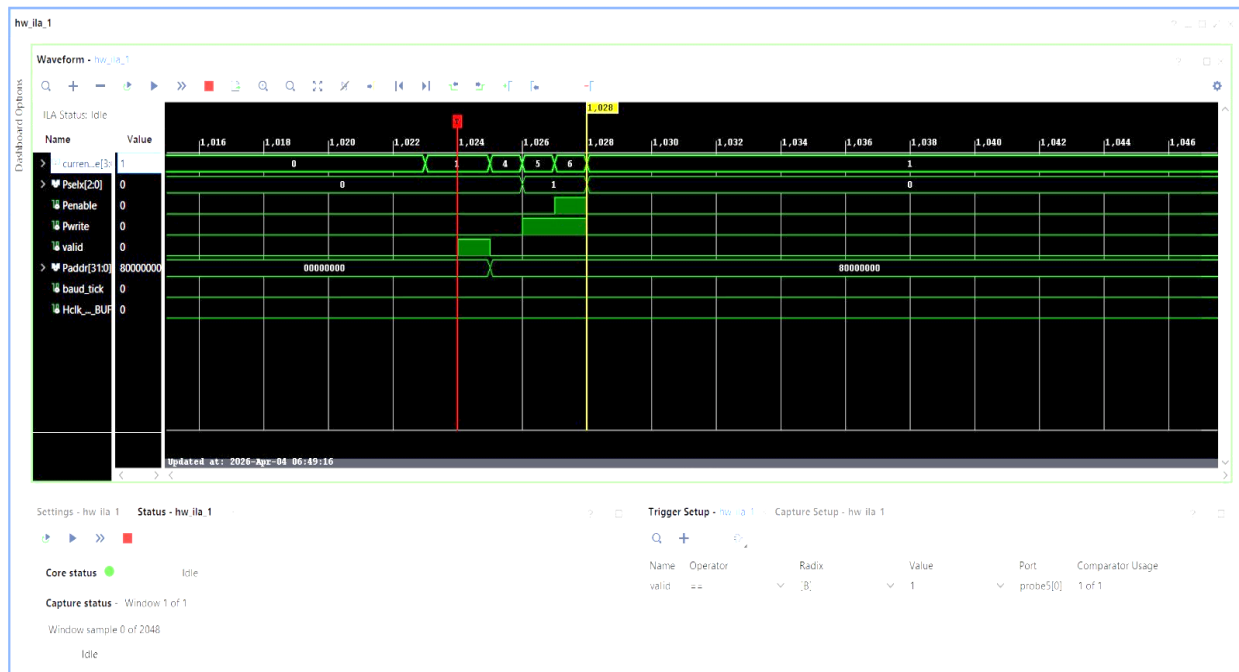


Fig 4.1.10 LED Write Operation

This ILA capture shows a valid APB write operation. The FSM moves from IDLE to WAIT, WRITE, and WENABLE.

Pselx = 001 selects the LED peripheral,

Pwrite = 1 confirms a write transaction,

Paddr = 0x80000000 shows the LED target address

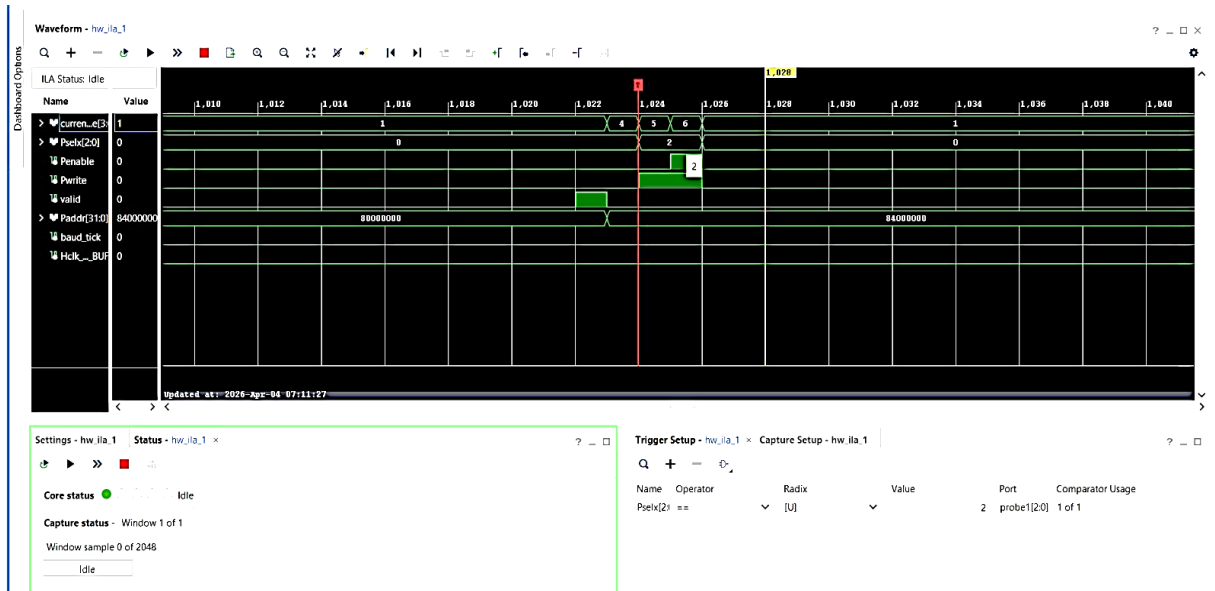


Fig 4.1.11 Timer Write Operation

This ILA capture shows a valid APB write operation. The FSM moves from IDLE to WWAIT, WRITE, and WENABLE.

Pselx = 010 selects the TIMER peripheral,

Pwrite = 1 confirms a write transaction,

Paddr = 0x84000000 shows the TIMER target address

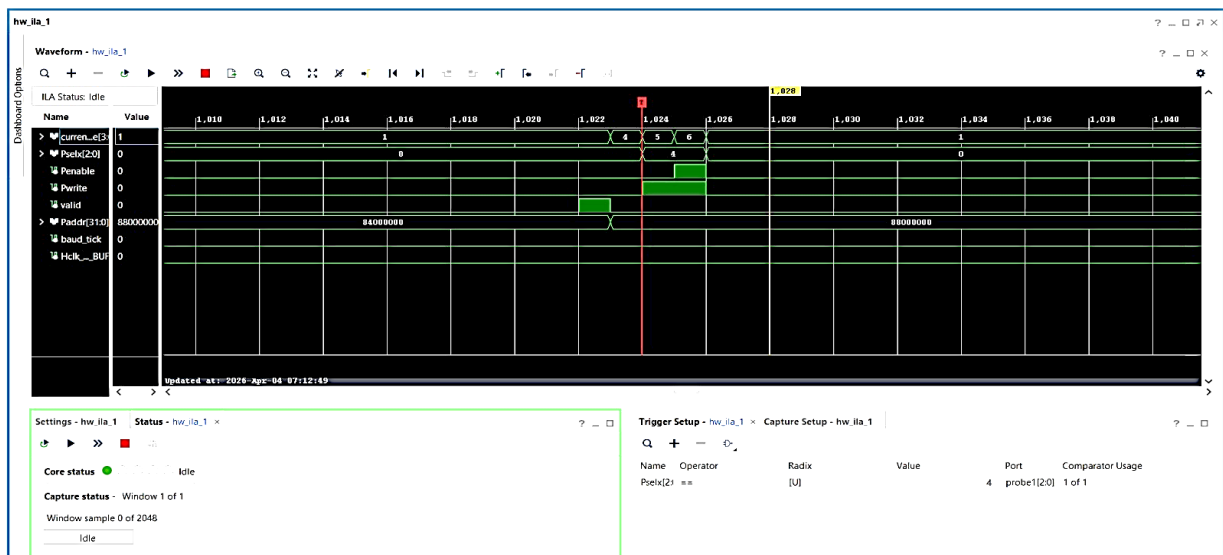


Fig 4.1.12 UART Write Operation

This ILA capture shows a valid APB write operation. The FSM moves from IDLE to WWAIT, WRITE, and WENABLE.

Pselx = 100 selects the UART peripheral,

Pwrite = 1 confirms a write transaction,

Paddr = 0x88000000 shows the UART target address

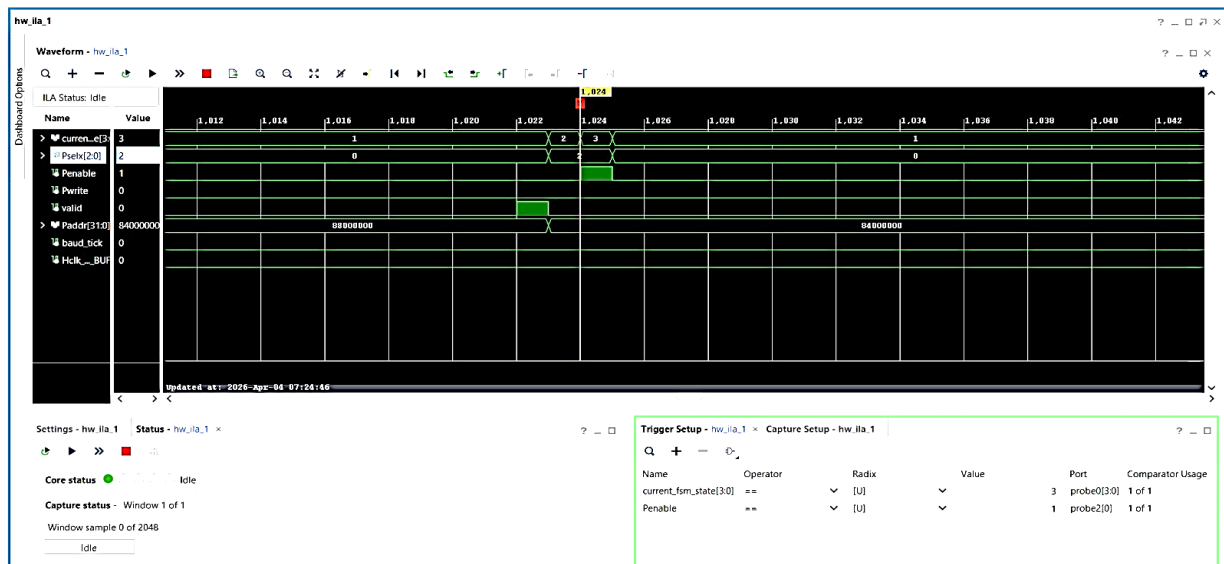


Fig 4.1.13 Read Operation

This ILA capture shows the APB read operation from the timer peripheral.

The FSM moves from READ to RENABLE

Pwrite remains low, Pselx = 010 selects the timer,

Paddr = 0x84000000 confirms the timer read address.

CHAPTER-5

ADVANTAGES AND APPLICATIONS

5.1 Advantages

5.1.1 Efficient Protocol Conversion

One of the major advantages of this project is its ability to efficiently perform protocol conversion between the Advanced High-performance Bus (AHB) and the Advanced Peripheral Bus (APB). These two buses operate with different speeds, signal structures, and communication mechanisms. The AHB bus supports high-speed data transfer with pipelining and burst operations, whereas the APB bus is designed for low-speed, low-power peripheral communication. The bridge ensures seamless translation of AHB transactions into APB-compatible signals without loss of data or synchronization issues. The use of a finite state machine (FSM) in the APB controller ensures proper sequencing of operations such as setup and enable phases. This guarantees that data transfer between the two domains is accurate and reliable. Additionally, the bridge handles control signals like HREADY and PENABLE effectively, ensuring proper handshake and synchronization[13]. This efficient protocol conversion reduces system complexity by eliminating the need for separate communication interfaces for each peripheral. It also improves overall system performance by enabling smooth interaction between high-speed processors and low-speed devices. Therefore, the bridge plays a critical role in ensuring efficient communication in modern digital systems.

5.1.2 Modular Design Architecture

The project follows a modular design approach, which is a significant advantage in digital system development. The entire system is divided into independent modules such as AHB master generator, AHB slave interface, APB controller, LED controller, timer module, UART, pattern generator, and seven-segment display controller. Each module is designed, tested, and verified separately before being integrated into the complete system. This modularity simplifies the design process and makes debugging easier, as issues can be identified and resolved within individual modules. It also enhances scalability, allowing additional peripherals or functionalities to be added without affecting the existing system. For example, new modules such as SPI, I2C, or ADC can be easily integrated into the APB bus. Furthermore, modular design promotes code reusability, as individual modules can be reused in other projects with minimal modification. It also improves maintainability, making it easier to update or upgrade specific components [14]. Overall, the modular

architecture increases design flexibility, reduces development time, and improves system reliability.

5.1.3 FPGA-Based Real-Time Implementation

Another important advantage of this project is its implementation on an FPGA platform, specifically the Basys 3 board. FPGA-based implementation allows the design to be tested and verified in real time, providing practical validation beyond simulation. This ensures that the system works correctly under actual operating conditions. The FPGA provides flexibility, as the design can be easily modified and reconfigured without requiring hardware changes. This is particularly useful during the development and debugging stages. The real-time outputs, such as LED patterns, UART communication, and seven-segment display, provide immediate feedback on system performance. Additionally, FPGA implementation helps identify issues related to timing, synchronization, and hardware constraints that may not be visible in simulation. It also enhances learning by providing hands-on experience with hardware design and verification. Overall, FPGA-based implementation improves design accuracy, reduces development risks, and ensures that the system is ready for real-world applications.

5.1.4 Support for Multiple Peripherals

The integration of multiple APB peripherals is a key advantage of this project. The system includes LED controller, timer module, UART transmitter, pattern generator, and seven-segment display controller, demonstrating a complete and functional multi-peripheral environment. The AHB to APB bridge uses address decoding logic to select the appropriate peripheral based on the input address. This allows the processor to control multiple devices through a single communication interface, reducing system complexity. Each peripheral performs a specific function, contributing to the overall functionality of the system. This multi-peripheral support reflects real-world SoC architectures, where multiple devices are connected to a single bus. It also enhances system flexibility, as additional peripherals can be added easily. The ability to handle multiple peripherals efficiently makes the system suitable for various applications, including embedded systems and industrial control.

5.1.5 Bidirectional Data Transfer

The project supports both read and write operations, enabling bidirectional data transfer between the AHB master and APB peripherals. This is an important advantage, as it allows the system to not only send control signals to peripherals but also receive data from them. In write mode, the processor configures peripherals such as LEDs, timer, and UART. In read mode, it retrieves data such as timer values or status information. This ensures complete communication between the processor and peripherals. Bidirectional communication is essential in real-world applications, where monitoring and feedback are required. It enhances system functionality and enables better control over peripheral operations. The successful implementation of both read and write operations demonstrates the reliability and completeness of the design.

5.1.6 Real-Time Debugging Using ILA

The integration of an Integrated Logic Analyzer (ILA) is a major advantage of this project. The ILA allows internal signals to be monitored in real time during FPGA operation, providing valuable insights into system behavior. Designers can observe signals such as FSM states, control signals, address lines, and data paths. This helps in verifying correct operation and identifying issues such as timing errors or incorrect state transitions. The ability to debug the system internally without external tools simplifies the development process. Real-time debugging improves design accuracy and reduces development time. It also enhances understanding of system operation, making it easier to optimize performance. Therefore, the use of ILA significantly improves the reliability and efficiency of the design.

5.1.7 Scalable and Flexible Architecture

The project is designed with scalability and flexibility in mind, allowing it to be extended for more complex applications. The modular architecture and address-based peripheral selection make it easy to add new peripherals or modify existing ones. For example, advanced communication modules such as SPI, I2C, or can be integrated into the system. Similarly, additional features such as interrupt handling or multiple AHB masters can be implemented to enhance functionality. This scalability makes the design suitable for a wide range of applications, from simple embedded systems to complex SoC architectures. It also ensures that the system can evolve with changing requirements, making it future-proof.

5.1.8 Reduced Power Consumption

The use of APB for peripheral communication contributes to reduced power consumption, which is an important advantage in modern digital systems. APB is designed for low-speed and low-power operation, making it ideal for connecting peripheral devices. By using the AHB to APB bridge, high-speed operations are limited to the AHB bus, while low-power peripherals operate on the APB bus[15]. This separation helps optimize power usage and improves overall system efficiency. Reduced power consumption is particularly important in battery-operated devices and embedded systems. It extends battery life and reduces heat generation, improving system reliability.

5.1.9 Improved System Organization

The AHB to APB bridge improves system organization by providing a structured communication framework. It separates high-speed and low-speed operations, making the system easier to design and manage. The use of address decoding and modular design ensures that each peripheral is accessed in an organized manner. This reduces design complexity and improves readability of the system architecture. A well-organized system is easier to debug, maintain, and upgrade. It also improves performance by ensuring efficient communication between components.

5.1.10 Strong Foundation for SoC Design

This project provides a strong foundation for understanding System-on-Chip (SoC) design concepts. It demonstrates key aspects such as bus architecture, protocol conversion, FSM design, and peripheral interfacing. The practical implementation on FPGA enhances learning and provides hands-on experience with real-world design challenges. It prepares students for advanced topics in VLSI design and embedded systems. Overall, this project serves as an excellent learning platform and a stepping stone for future work in digital system design and SoC development.

5.1.11 Enhanced Multi-Master Support and Arbitration Logic

A significant future enhancement involves integrating an **AHB Bus Matrix or Multi-Master Arbiter**. In the current design, the system relies on a single AHB Master; however, modern SoCs frequently utilize multiple masters, such as a CPU, a Direct Memory Access (DMA) controller, and a high-speed communication MAC. Implementing **Round-Robin or**

Fixed-Priority Arbitration would allow the bridge to manage simultaneous requests from multiple sources. This transition would significantly increase system throughput, allowing data-intensive tasks (like moving UART logs to memory via DMA) to occur in the background without stalling the main processor, thereby mimicking the efficiency of professional-grade ARM-based architectures.

5.1.12 Integration of Industrial Communication Protocols

The modularity of the APB interface provides an ideal foundation for extending the system's peripheral library to include a wider array of industrial standards. Future iterations could integrate an **SPI (Serial Peripheral Interface)** for high-speed sensor data acquisition and an **I2C (Inter-Integrated Circuit)** master for managing low-speed system components like power regulators or EEPROMs. Additionally, adding a **PWM (Pulse Width Modulation)** module would allow the Basys 3 to drive motor controllers, while an **ADC (Analog-to-Digital Converter)** bridge would enable the system to process real-world analog signals. This expansion would transform the bridge from a simple protocol translator into a comprehensive embedded control hub.

5.1.13 Performance Advantage: Architectural Isolation and Timing Integrity

One of the primary advantages of this bridge is its role as an **architectural firewall** that ensures system-wide timing integrity. By isolating the high-speed **100 MHz AHB domain** from the slower peripheral logic, the bridge prevents high-frequency switching noise and timing violations from propagating into sensitive peripheral circuits. This decoupling allows the AHB Master to issue a command and immediately return to high-performance tasks while the bridge independently handles the multi-cycle handshaking required by the APB protocol. This not only optimizes the processor's duty cycle but also reduces the overall dynamic power consumption by localizing high-speed transitions to only the necessary sections of the FPGA fabric.

5.2 Applications

5.2.1 System-on-Chip (SoC) Design

The AHB to APB bridge is a fundamental building block in System-on-Chip (SoC) architecture, where multiple functional components are integrated onto a single chip to perform complex operations efficiently. In modern SoC designs, high-speed components such as processors, memory units, and Direct Memory Access (DMA) controllers communicate using high-performance buses like the Advanced High-performance Bus (AHB). At the same time, low-speed peripherals such as timers, UART modules, LED controllers, and input/output interfaces operate using simpler and low-power buses like the Advanced Peripheral Bus (APB). Since these two buses differ significantly in terms of speed, complexity, and protocol structure, an interface is required to enable seamless communication between them. This is achieved using the AHB to APB bridge[16]. The bridge performs protocol conversion by translating AHB transactions into APB-compatible signals while maintaining synchronization and data integrity. It ensures that high-speed data from the AHB domain is correctly transferred to low-speed APB peripherals without timing issues or data loss. In this project, the integration of peripherals such as LED controller, timer module, pattern generator, UART, and seven-segment display represents a simplified SoC model. Each peripheral is assigned a unique address range, and the bridge uses address decoding logic to select the appropriate device. The implementation of a finite state machine (FSM) in the APB controller ensures proper sequencing of operations, including setup and enable phases required for APB communication. The bridge also supports both read and write operations, enabling bidirectional data transfer. This is essential in SoC systems, where the processor not only controls peripherals but also reads their status or data. The FPGA-based implementation further validates the design by demonstrating real-time operation. Outputs such as LED patterns, UART communication, and seven-segment display provide clear evidence of correct functionality. Thus, this application highlights the importance of the AHB to APB bridge in enabling efficient, scalable, and reliable communication in SoC architectures.

5.2.2 Embedded Systems

The AHB to APB bridge plays a significant role in embedded systems, where efficient communication between a central processing unit and multiple peripheral devices is essential

for system functionality. Embedded systems are designed to perform specific tasks and are widely used in applications such as home automation, automotive control, medical devices, and consumer electronics. In these systems, the processor typically operates at high speed, while peripherals such as sensors, actuators, timers, displays, and communication modules operate at lower speeds. The difference in operating speeds and protocol requirements makes direct communication challenging, and this is where the AHB to APB bridge becomes crucial. The bridge enables seamless interaction between high-speed and low-speed components by converting AHB transactions into APB protocol signals. This ensures that the processor can efficiently control and monitor peripheral devices without compromising system performance. In this project, peripherals such as LED controller, timer module, UART transmitter, and pattern generator represent typical embedded system components[17]. The LED controller can be used for visual status indication, the timer module for generating delays or scheduling tasks, and the UART for serial communication with external devices. The modular design of the system allows each peripheral to operate independently while being controlled through a unified communication framework. This improves system organization and scalability, making it easier to add new peripherals or modify existing ones. The use of FPGA implementation provides a practical platform for testing and validating embedded system designs in real time. Outputs such as LED patterns and UART transmission provide immediate feedback, helping designers verify correct functionality. Additionally, the integration of an Integrated Logic Analyzer (ILA) enables real-time monitoring of internal signals, which is useful for debugging and optimizing system performance. Overall, the AHB to APB bridge enhances communication efficiency, reduces design complexity, and supports scalable architecture in embedded systems, making it a vital component in modern digital applications.

5.2.3 FPGA-Based Prototyping and Hardware Validation

The AHB to APB bridge is widely used in FPGA-based prototyping, which is an essential stage in modern digital system design. Field Programmable Gate Arrays (FPGAs) provide a flexible and reconfigurable platform for implementing and testing hardware designs before final production. This project demonstrates the implementation of the AHB to APB bridge on the Basys 3 FPGA board using Verilog HDL, allowing real-time validation of system functionality. In FPGA prototyping, the bridge serves as the communication backbone that connects the AHB master interface with multiple APB peripherals. By integrating modules such as LED controller, timer, UART, pattern generator, and seven-segment display, the

design becomes a complete and functional system. This enables designers to test the interaction between different modules and verify correct data transfer under various conditions. One of the major advantages of FPGA-based prototyping is the ability to observe system behavior directly through hardware outputs. For example, LED patterns indicate successful data transfer, UART communication confirms proper serial data transmission, and the seven-segment display provides real-time status information. These outputs help in verifying the correctness of the design and identifying potential issues. The use of an Integrated Logic Analyzer (ILA) further enhances this application by allowing internal signals to be monitored during operation. This helps in debugging complex issues such as timing violations, incorrect state transitions, and protocol mismatches. FPGA prototyping reduces development time and cost by enabling early detection of design errors. This application is widely used in industry for validating complex digital systems, including SoCs, communication protocols, and embedded controllers. Therefore, the AHB to APB bridge plays a crucial role in FPGA-based hardware validation and prototyping.

5.2.4 Industrial Control Systems

The AHB to APB bridge architecture is highly suitable for industrial control systems, where a central processing unit is responsible for managing and monitoring multiple peripheral devices. Industrial environments typically involve complex operations such as automation, process control, machine monitoring, and data acquisition. These systems require reliable communication between high-speed controllers and low-speed peripherals such as sensors, actuators, timers, and communication interfaces. The AHB to APB bridge provides an efficient solution by enabling structured communication between these components. In this project, the timer module can be used for generating precise delays and scheduling operations, which are critical in industrial automation processes. The LED controller can serve as a status indicator, displaying the operational state of machines or processes. The UART module can be used to transmit diagnostic or monitoring data to external systems, enabling real-time supervision. The pattern generator can simulate control signals or testing sequences, which are useful in validating system functionality [18]. The modular design of the system ensures flexibility, allowing new peripherals to be added without affecting the existing architecture. Address decoding logic enables the controller to select specific devices based on operational requirements, improving system efficiency. FPGA implementation provides robustness and adaptability, which are essential in industrial applications where reliability is critical. Furthermore, the integration of an Integrated Logic Analyzer (ILA)

allows engineers to monitor internal signals in real time, helping in debugging and optimizing system performance. This is particularly useful in industrial settings, where quick identification and resolution of issues are necessary to minimize downtime. Overall, the AHB to APB bridge provides a reliable and scalable communication framework for industrial control systems.

5.2.5 Real-Time Debugging and Verification Systems

The AHB to APB bridge system is highly applicable in real-time debugging and verification of digital hardware designs. In modern digital systems, verifying correct functionality after synthesis and implementation is a challenging task. Simulation alone cannot always capture real hardware issues such as timing mismatches, signal delays, and synchronization errors. This project addresses these challenges by integrating an Integrated Logic Analyzer (ILA), which enables real-time monitoring of internal FPGA signals. The ILA captures important signals such as AHB control lines, APB enable signals, address buses, data paths, and finite state machine (FSM) transitions. By analyzing these signals, designers can verify the correctness of protocol conversion and ensure that the bridge operates as expected. For example, designers can check whether the FSM transitions occur in the correct sequence or whether the correct peripheral is selected during a transaction. The use of external outputs such as LEDs, UART communication, and seven-segment display further enhances debugging by providing visible indicators of system behavior. These outputs help confirm that data transfer is occurring correctly and that peripherals are functioning as intended. The combination of internal signal monitoring and external outputs provides a comprehensive debugging environment. This application is widely used in FPGA design labs, research environments, and industrial development processes. It helps designers identify errors quickly, improve system reliability, and optimize performance. Thus, the AHB to APB bridge serves as an effective platform for real-time debugging and verification of digital systems.

5.2.6 UART Communication Systems

The inclusion of a UART module in this project makes the AHB to APB bridge highly applicable in communication systems, particularly those involving serial data transmission. UART (Universal Asynchronous Receiver/Transmitter) is one of the most widely used communication protocols in embedded systems due to its simplicity, reliability, and low

cost. It enables data exchange between digital devices without requiring a clock signal, making it suitable for many practical applications. In this project, the AHB to APB bridge allows the processor or AHB master to control the UART peripheral through APB. Data can be written to the UART module, which then transmits it serially to an external device such as a computer or terminal. This is demonstrated by transmitting the string “HELLO,” which confirms correct operation of the communication system. The bridge ensures proper synchronization between the high-speed AHB bus and the UART module, which operates at a lower speed. UART communication is widely used in applications such as debugging, data logging, sensor communication, and device control. The bridge-based architecture allows the UART module to be easily integrated with other peripherals, creating a complete communication system. Additionally, the modular design enables the UART to be replaced or upgraded without affecting other components. The FPGA implementation provides real-time verification of UART functionality, while the ILA helps monitor internal signals related to data transmission. This ensures reliable communication and helps identify any issues during operation. Therefore, the AHB to APB bridge is highly useful in UART-based communication systems.

5.2.7 LED Control and Display Systems

The LED controller integrated into the AHB to APB bridge system enables its application in LED control and display systems. LEDs are commonly used in digital systems for visual indication of status, data, and system operation. They provide a simple and effective way to observe system behavior without requiring complex interfaces. In this project, the LED controller receives data from the AHB master through the APB bus and displays it on LEDs. The pattern generator and timer modules can be used to create dynamic LED patterns such as blinking, shifting, or rotating sequences. These patterns provide visual feedback that confirms correct data transfer and system operation. LED-based systems are widely used in applications such as status indicators, display panels, and testing environments. The AHB to APB bridge allows the processor to control LED outputs through a structured communication framework, improving system organization and scalability. The modular design also allows additional display modules to be integrated easily.

The FPGA implementation provides real-time visualization of LED outputs, making it easier to verify system functionality. The use of ILA further enhances debugging by allowing internal signals to be monitored. This helps ensure that the LED controller operates correctly and responds to control signals as expected. Overall, the AHB to APB bridge enables

efficient control of LED-based display systems, making it useful in both practical applications and educational environments.

5.2.8 Pattern Generation Systems

The pattern generator module makes the AHB to APB bridge system suitable for pattern generation applications. Pattern generators are widely used in digital systems for testing, demonstration, and signal generation purposes. They produce sequences of outputs that can be used to verify system functionality or create visual effects. In this project, the pattern generator produces LED patterns such as shifting or blinking sequences. These patterns are controlled through the APB bus, allowing the processor to modify them dynamically. The timer module can be used to control the speed of pattern generation, providing flexibility in output behavior. Pattern generation is useful in applications such as digital testing, signal analysis, and educational demonstrations. It helps verify that data is being transferred correctly and that control signals are functioning as expected. The AHB to APB bridge enables seamless communication between the processor and the pattern generator, ensuring accurate control of output sequences. The FPGA implementation allows real-time observation of patterns, while the ILA provides insight into internal signal behavior. This combination makes it easier to debug and optimize the system. Therefore, the AHB to APB bridge is highly useful in pattern generation systems.

5.2.9 Timer-Based Applications

The timer module integrated into the system enables the AHB to APB bridge to be used in timer-based applications. Timers are essential components in digital systems, as they provide precise control over timing and synchronization of operations. In this project, the timer module generates periodic signals that can be used to control the speed of LED patterns or other operations. The processor can configure the timer through the APB bus and read back its value for verification. This demonstrates how timing peripherals can be integrated into a bus-based system. Timer-based applications include delay generation, scheduling, event counting, and synchronization of processes. The AHB to APB bridge ensures that the processor can interact with the timer module efficiently, enabling accurate control of system timing. The FPGA implementation provides real-time verification of timer functionality, while the ILA helps monitor internal signals. This ensures reliable operation and accurate timing control. Thus, the bridge is highly useful in timer-based applications.

5.2.10 Educational and Academic Applications

The AHB to APB bridge project is highly useful in educational and academic environments for teaching digital system design concepts. It provides a practical example of bus architecture, protocol conversion, and FPGA implementation. Students can use this project to understand how different components in a digital system communicate with each other. The integration of peripherals such as LED controller, timer, UART, and pattern generator provides a comprehensive learning experience. The FPGA implementation allows students to observe real-time system behavior, while the ILA helps them understand internal signal operations. This hands-on approach enhances learning and helps students develop practical skills.

5.2.11 Consumer Electronics

The AHB to APB bridge is widely used in consumer electronics, where processors communicate with peripheral devices such as displays, communication modules, and control interfaces. This project demonstrates a similar architecture, making it useful for understanding how consumer devices operate internally. The bridge enables efficient communication between components, improving system performance.

5.2.12 Peripheral Subsystem Development

The implementation of the **AHB-to-APB bridge** provides a robust and highly scalable development platform for the rapid integration of specialized hardware modules. By adhering to the standardized **AMBA APB protocol**, designers can seamlessly extend the system's functionality with peripherals such as **SPI controllers, I2C interfaces, Analog-to-Digital Converters (ADC), and Pulse Width Modulators (PWM)**. This modular architecture simplifies the design cycle, as new components can be added to the existing memory map without necessitating a complete overhaul of the high-speed **AHB Master** logic.

Furthermore, the bridge acts as a protective buffer, ensuring that the timing requirements of new, slower peripherals are met without compromising the stability of the primary system bus. This approach is particularly advantageous in hardware validation and testing environments, where modularity allows for the isolated debugging of individual components. Consequently, the project serves as a foundational template for building complex, energy-efficient **System-on-Chip (SoC)** environments, where a diverse array of communication and control interfaces must coexist and interact under a unified bus management system.

5.2.13 Digital System Testing Platforms

The project serves as a highly versatile **Hardware-in-the-Loop (HIL)** testing platform, specifically engineered for verifying the timing and protocol integrity between diverse SoC modules. By capturing internal data flows and handshaking signals through the **Integrated Logic Analyzer (ILA)**, it allows designers to identify bottlenecks or synchronization errors in real-time. This capability is invaluable in hardware validation environments, as it provides a low-cost, high-reliability method for testing custom peripherals before they are integrated into more complex, multi-layered architectures. Thus, the bridge acts as a controlled gateway for analyzing how high-speed system commands are translated into stable, peripheral-level executions on physical silicon.

5.2.14 Communication Interface Systems

The bridge serves as a critical communication gateway by enabling the seamless integration of standardized interfaces like **UART** into high-performance system architectures. This integration is particularly useful in embedded systems requiring robust data exchange between a high-speed processor and external monitoring tools. By utilizing the **AMBA APB protocol**, the bridge provides a stable, low-complexity environment for the UART module to serialize data without being overwhelmed by the high-frequency bursts of the **AHB master**.

Furthermore, this setup allows for real-time system telemetry, where internal status registers can be transmitted to a PC for analysis. This capability makes the design an ideal foundation for industrial control systems and IoT gateways, where reliable data logging and external communication are mandatory requirements. Through this hardware-validated approach, the bridge proves that complex bus protocols can be effectively throttled to maintain high-integrity data paths across diverse communication standards.

5.2.15 Research and Development

The project serves as a highly versatile **Hardware-in-the-Loop (HIL)** testing platform, specifically engineered for verifying the timing and protocol integrity between diverse SoC modules. By capturing internal data flows and handshaking signals through the **Integrated Logic Analyzer (ILA)**, it allows designers to identify bottlenecks or synchronization errors in real-time. This capability is invaluable in hardware validation environments, as it provides a low-cost, high-reliability method for testing custom peripherals before they are integrated into

more complex, multi-layered architectures. Thus, the bridge acts as a controlled gateway for analyzing how high-speed system commands are translated into stable, peripheral-level executions on physical silicon.

5.2.16 Multi-Master Support and Advanced Arbitration

One of the most significant enhancements would be the integration of an **AHB Bus Matrix or Arbiter**. Currently, the system relies on a single AHB Master; however, modern SoCs often feature multiple masters, such as a CPU, a Direct Memory Access (DMA) controller, and a high-speed Ethernet MAC, all competing for the same peripheral resources. Implementing a **Round-Robin or Fixed-Priority Arbitration** logic would allow the bridge to manage simultaneous requests, significantly increasing the system's total throughput and allowing for background data transfers without CPU intervention.

5.2.17 Integration of High-Speed and Industrial Peripherals

The modular nature of the APB side provides an ideal foundation for adding a diverse range of industrial-standard peripherals. Future versions of this project could include an **SPI (Serial Peripheral Interface) Controller** for interfacing with external sensors or flash memory, and an **I2C (Inter-Integrated Circuit) Master** for communication with low-speed power management ICs. Furthermore, the addition of a **PWM (Pulse Width Modulation)** module would enable the FPGA to control motor drivers or dimming circuits, while an **ADC (Analog-to-Digital Converter)** interface would allow the bridge to process real-world environmental data, transforming the project into a complete embedded control system.

5.2.18 Interrupt Management and Low-Power Optimization

To improve the efficiency of the software-hardware interface, a **Vectored Interrupt Controller (VIC)** could be implemented. In the current design, the master must "poll" the UART or Timer to check for status updates. By adding interrupt lines from the APB peripherals back to the AHB domain, the master can remain in a low-power "sleep" state until a peripheral requires attention. Additionally, **Clock Gating** techniques could be applied to the APB Controller FSM, where the clock is only supplied to the bridge logic when a valid transaction is detected, drastically reducing the dynamic power consumption of the Basys 3 implementation.

5.2.19 Transition to AXI and Advanced Bus Protocols

As the design matures, the bridge logic could be adapted to the **AXI (Advanced eXtensible Interface)** protocol, which is the current industry standard for high-performance ARM-based systems. Unlike AHB, AXI supports out-of-order transactions and separate channels for read and write data, allowing for even higher levels of concurrency. Moving from an **AHB-to-APB** model to an **AXI-to-APB** bridge would align the project with contemporary professional FPGA design flows used in Xilinx Vivado and Altera Quartus environments, making it a powerful template for high-speed data acquisition and processing systems.

CHAPTER-6

CONCLUSION

The successful design and FPGA implementation of the **AHB-to-APB Bridge** using **Verilog HDL** represents a sophisticated transition from theoretical **AMBA (Advanced Microcontroller Bus Architecture)** specifications to a fully functional, physical hardware prototype. By leveraging the **Xilinx Artix-7 FPGA** on the **Basys 3** platform, the project demonstrates a high-integrity solution for the fundamental System-on-Chip (SoC) challenge of interfacing high-bandwidth, pipelined system buses with low-power, non-pipelined peripheral domains. At the core of this achievement is the **9-state Finite State Machine (FSM)**, which serves as the "protocol translator" by meticulously managing the timing disparities between the **100 MHz AHB Master** and the slower peripheral logic. This FSM ensures that every **WRITE** and **READ** transaction adheres to strict setup and hold time requirements—specifically through the use of **WWAIT**, **WRITE**, and **WENABLE** states—thereby preventing data corruption or bus contention that often plagues high-speed digital designs. The **AHB Slave Interface** further strengthens this architecture by acting as a high-speed buffer, capturing , , and signals in real-time, while the integrated **Address Decoder** utilizes a one-hot tempselx encoding scheme to isolate the **LED Controller**, **System Timer**, and **UART** modules within the memory map.

The practical utility of this bridge is validated through its multi-peripheral integration, where the **System Timer** generates periodic "ticks" to drive a synchronized **LED Pattern Module**, and the **UART peripheral** successfully serializes ASCII data for external observation on **PuTTY**. This end-to-end communication path—from the master's initial write sequence to the serial bitstream—proves the reliability of the bridge's internal handshaking and data-routing logic. Furthermore, the inclusion of the **Integrated Logic Analyzer (ILA)** and the **four-digit seven-segment display** provides a dual-layer verification system, allowing for both nanosecond-level waveform analysis within the **Vivado Design Suite** and immediate human-readable status updates on the FPGA hardware. By mirroring internal registers like `data_byte` and `op_write` onto the physical display, the project bridges the gap between abstract HDL code and tangible silicon behavior. Ultimately, this implementation offers a scalable and modular foundation for modern embedded systems, proving that complex, high-frequency bus architectures can be effectively throttled and managed to support a diverse array of communication and control interfaces in an energy-efficient, robust, and industry-standard SoC environment.

REFERENCE

- [1] K. Rawat; Kanika Sahni; S. Pandey. "RTL Implementation for AMBA ASB APB Protocol at System on Chip Level" 2020 IEEE International Conference on System-on-Chip Design and Implementation (SOCDI).
- [2] K. Rawat; Kanika Sahni; S. Pandey. "Design of AMBA APB Bridge with Reset Controller for Efficient Power Consumption" 2020 IEEE International Conference on Sustainable Computing in Electrical and Electronics Engineering (ICSCEE).
- [3] B. N. Manu P.; Prabhavathi. "Design and Implementation of AMBA ASB APB Bridge" 2019 IEEE International Conference on Embedded Systems (ICES).
- [4] Chenghai Ma; Zhijun Liu; Xiaoyue Ma. "Design and Implementation of APB Bridge Based on AMBA 4.0" 2018 IEEE International Conference on Microelectronics (ICM).
- [5] L. Deeksha; B. Shivakumar. "Effective Design and Implementation of AMBA AHB Bus Protocol using Verilog" 2018 IEEE International Conference on Advances in Computing, Communications and Informatics (ICACCI).
- [6] Sajidha Thabassum B et al., "Design and Implementation of AHB To APB Bridge Using Verilog," IJCRTIJCRT, vol. 13, no. 1, Jan. 2025.
- [7] D. Venkanna Babu et al., "Design and Implementation of Amba Based AHB to APB Bridge Protocol," IJSRSET, vol. 11, no. 2, pp. 321-332, Apr. 2024.
- [8] R. Dholiya, "Design of AHB to APB Bridge," BVM Engineering Project Report, 2020.
- [9] K. V. and B. S. Appasaheb, "Implementation and Verification of AHB 5.0 to APB Bridge," J. Harbin Eng. Univ., vol. 45, no. 9, Sep. 2024.
- [10] Bellerimath P. S. et al., "Design and Implementation of AHB to APB Bridge using Verilog," IJEMRIJEMR, Mar. 2025.
- [11] Padmaprabha Jain; Satheesh Rao. "Design and Verification of Advanced Microcontroller Bus Architecture-Advanced Peripheral Bus (AMBA-APB) Protocol" 2020 IEEE International Conference on Recent Advances in Communication and Computing (ICRACC).
- [12] Jaehoon Song, Student member, IEEE, Hyunbean Yi, Member, IEEE, Juhee Han, and Sungju Park, Member, IEEE, "An Efficient SOC Test Technique by Reusing On/Off-Chip Bus Bridge" IEEE Transactions on Circuits and Systems-I: Regular Papers, Vol,56, No.3, March2009.
- [13] Sangik Choi and Shinwook Kang, Mobile Samsung Electronics Co., Ltd, "Implementation of an On-Chip Bus Bridge between Heterogeneous Buses with Different Clock Frequencies". IEEE, IDEAS'05,1098-8068/2005.

- [14] Flynn, D. Adv. RISC Machines Ltd., Cambridge, “AMBA: enabling reusable on-chip designs”, IEEE Micro, Publication Date: Jul/Aug 1997.
- [15] Wang Zhonghai, Ye Yizheng, Wang Jinxing, and Yu Mingyan, “Designing AHB/PCI Bridge,”in Proceedings of 4th International Conference on ASIC, Oct 2001, pp.578-580.
- [16] A. Kharade; Jayashree V. “VLSI Design of AMBA Based AHB2APB Bridge” 2021 IEEE International Conference on Electronics and Communication Systems (ICECS).
- [17] M. K. Kumar; Amritha Sajja. “Design and FPGA Implementation of AMBA APB Bridge with Clock Skew Minimization Technique” 2021 IEEE International Symposium on VLSI Design and Test (VDATE).
- [18] Padmaprabha Jain; Satheesh Rao. “Design and Verification of Advanced Microcontroller Bus Architecture-Advanced Peripheral Bus (AMBA-APB) Protocol” 2020 IEEE International Conference on Recent Advances in Communication and Computing (ICRACC).

ANNEXURE

1. Implementation

1.1 Software Tools Used

- Xilinx Vivado
- Putty

1.2 AHB to APB Bridge Design Code

AHB Generator module

```
module ahb_gen(
    input clk, rst,
    input Hreadyin,
    input [31:0] Hrdata,
    output reg Hwrite,
    output reg [1:0] Htrans,
    output reg [31:0] Haddr,
    output reg [31:0] Hwdata
);

    localparam [5:0]
        LED_ADDR = 6'd0,
        LED_DATA = 6'd1,
        WAIT_0 = 6'd2,
        TIMER_ADDR = 6'd3,
        TIMER_DATA = 6'd4,
        WAIT_1 = 6'd5,
        H_ADDR = 6'd6,
        H_DATA = 6'd7,
        WAIT_2 = 6'd8,
        E_ADDR = 6'd9,
        E_DATA = 6'd10,
        WAIT_3 = 6'd11,
        L1_ADDR = 6'd12,
        L1_DATA = 6'd13,
        WAIT_4 = 6'd14,
        L2_ADDR = 6'd15,
        L2_DATA = 6'd16,
        WAIT_5 = 6'd17,
        O_ADDR = 6'd18,
        O_DATA = 6'd19,
        WAIT_6 = 6'd20,
        READ_ADDR = 6'd21,
        READ_CAPTURE = 6'd22,
        DONE = 6'd23;

    localparam [25:0]
```

```
        DEMO_DELAY =
        26'd50_000_000;
```

```
        reg [5:0] state;
        reg [25:0] delay_count;
        reg [31:0] readback_data;

    always @(posedge clk)
        begin
            if (rst)
                begin
                    state <= LED_ADDR;
                    delay_count <= 0;
                    readback_data <= 0;
                    Hwrite <= 1'b0;
                    Htrans <= 2'b00;
                    Haddr <= 32'd0;
                    Hwdata <= 32'd0;
                end

            else if (delay_count != 0)
                begin
                    delay_count <= delay_count -
                    1'b1;
                    Hwrite <= 1'b0;
                    Htrans <= 2'b00;
                    Haddr <= 32'd0;
                    Hwdata <= 32'd0;
                end

            else if (Hreadyin)
                begin
                    case (state)

                        LED_ADDR:
                            begin
                                Haddr <= 32'h8000_0000;
                                Hwdata <= 32'h0000_000A;
                                Hwrite <= 1'b1;
```

```

        Htrans <= 2'b10;
        state <= LED_DATA;
    end
LED_DATA:
    begin
        Haddr <= 32'h8000_0000;
        Hwdata <= 32'h0000_000A;
        Hwrite <= 1'b1;
        Htrans <= 2'b00;
        state <= WAIT_0;
    end
end

```

```

WAIT_0:
    begin
        Haddr <= 32'd0;
        Hwdata <= 32'd0;
        Hwrite <= 1'b0;
        Htrans <= 2'b00;
        delay_count <=
DEMO_DELAY;
        state <= TIMER_ADDR;
    end
end

```

```

TIMER_ADDR:
    begin
        Haddr <= 32'h8400_0000;
        Hwdata <= 32'h02FAF000;
        Hwrite <= 1'b1;
        Htrans <= 2'b10;
        state <= TIMER_DATA;
    end
end

```

```

TIMER_DATA:
    begin
        Haddr <= 32'h8400_0000;
        Hwdata <= 32'h02FAF000;
        Hwrite <= 1'b1;
        Htrans <= 2'b00;
        state <= WAIT_1;
    end
end

```

```

WAIT_1:
    begin
        Haddr <= 32'd0;
        Hwdata <= 32'd0;
        Hwrite <= 1'b0;
        Htrans <= 2'b00;
        delay_count <=
DEMO_DELAY;
        state <= H_ADDR;
    end
end

```

```

    end
H_ADDR:
    begin
        Haddr <= 32'h8800_0000;
        Hwdata <= 32'h0000_0048;
        Hwrite <= 1'b1;
        Htrans <= 2'b10;
        state <= H_DATA;
    end
end

```

```

H_DATA:
    begin
        Haddr <= 32'h8800_0000;
        Hwdata <= 32'h0000_0048;
        Hwrite <= 1'b1;
        Htrans <= 2'b00;
        state <= WAIT_2;
    end
end

```

```

WAIT_2:
    begin
        Haddr <= 32'd0;
        Hwdata <= 32'd0;
        Hwrite <= 1'b0;
        Htrans <= 2'b00;
        delay_count <=
DEMO_DELAY;
        state <= E_ADDR;
    end
end

```

```

E_ADDR:
    begin
        Haddr <= 32'h8800_0000;
        Hwdata <= 32'h0000_0045;
        Hwrite <= 1'b1;
        Htrans <= 2'b10;
        state <= E_DATA;
    end
end

```

```

E_DATA:
    begin
        Haddr <= 32'h8800_0000;
        Hwdata <= 32'h0000_0045;
        Hwrite <= 1'b1;
        Htrans <= 2'b00;
        state <= WAIT_3;
    end
end

```

```

WAIT_3:

```

```

begin
    Haddr <= 32'd0;
    Hwdata <= 32'd0;
    Hwrite <= 1'b0;
    Htrans <= 2'b00;
    delay_count <=
DEMO_DELAY;
    state <= L1_ADDR;
end

L1_ADDR:
begin
    Haddr <= 32'h8800_0000;
    Hwdata <= 32'h0000_004C;
    Hwrite <= 1'b1;
    Htrans <= 2'b10;
    state <= L1_DATA;
end

L1_DATA:
begin
    Haddr <= 32'h8800_0000;
    Hwdata <= 32'h0000_004C;
    Hwrite <= 1'b1;
    Htrans <= 2'b00;
    state <= WAIT_4;
end

WAIT_4:
begin
    Haddr <= 32'd0;
    Hwdata <= 32'd0;
    Hwrite <= 1'b0;
    Htrans <= 2'b00;
    delay_count <=
DEMO_DELAY;
    state <= L2_ADDR;
end

L2_ADDR:
begin
    Haddr <= 32'h8800_0000;
    Hwdata <= 32'h0000_004C;
    Hwrite <= 1'b1;
    Htrans <= 2'b10;
    state <= L2_DATA;
end

L2_DATA:

```

```

begin
    Haddr <= 32'h8800_0000;
    Hwdata <= 32'h0000_004C;
    Hwrite <= 1'b1;
    Htrans <= 2'b00;
    state <= WAIT_5;
end

WAIT_5:
begin
    Haddr <= 32'd0;
    Hwdata <= 32'd0;
    Hwrite <= 1'b0;
    Htrans <= 2'b00;
    delay_count <=
DEMO_DELAY;
    state <= O_ADDR;
end

O_ADDR:
begin
    Haddr <= 32'h8800_0000;
    Hwdata <= 32'h0000_004F;
    Hwrite <= 1'b1;
    Htrans <= 2'b10;
    state <= O_DATA;
end

O_DATA:
begin
    Haddr <= 32'h8800_0000;
    Hwdata <=
32'h0000_004F;
    Hwrite <= 1'b1;
    Htrans <= 2'b00;
    state <= WAIT_6;
end

WAIT_6:
begin
    Haddr <= 32'd0;
    Hwdata <= 32'd0;
    Hwrite <= 1'b0;
    Htrans <= 2'b00;
    delay_count <=
DEMO_DELAY;
    state <= READ_ADDR;
end

READ_ADDR:

```

```

begin
    Haddr <= 32'h8400_0000;
    Hwdata <= 32'd0;
    Hwrite <= 1'b0;
    Htrans <= 2'b10;
    state <=
READ_CAPTURE;
end

READ_CAPTURE:
begin
    Haddr <= 32'h8400_0000;
    Hwdata <= 32'd0;
    Hwrite <= 1'b0;
    Htrans <= 2'b00;
    readback_data <= Hrdata;
    delay_count <=
DEMO_DELAY;
    state <= DONE;
end

default:
begin
    Haddr <= 32'd0;
    Hwdata <= 32'd0;
    Hwrite <= 1'b0;
    Htrans <= 2'b00;
    readback_data <= Hrdata;
    state <= DONE;
end
endcase
end
end

endmodule

```

AHB slave interface

```

module ahb_slave_interface
( input Hclk, Hresetn, Hwrite,
  Hreadyin,
  input [1:0] Htrans,
  input [31:0] Haddr, Hwdata,
  output reg valid,

```

```

output reg [31:0] Haddr1,
  Hwdata1,
  output reg Hwritereg,
  output reg [2:0] tempselx
);
always @(posedge Hclk)
begin
    if (~Hresetn)
begin
    Haddr1 <= 0;
    Hwdata1 <= 0;
    Hwritereg <= 0;
    valid <= 1'b0;
end
else
begin
    valid <= 1'b0;
    if (Hreadyin && Htrans[1])
begin
    Haddr1 <= Haddr;
    Hwritereg <= Hwrite;
    if (Hwrite)
        Hwdata1 <= Hwdata;
    valid <= (Haddr >=
32'h8000_0000 &&
Haddr <
32'h8C00_0000);
end
end
end
// Address Decoding
always @(*)
begin
    if (Haddr1 >= 32'h8000_0000 && Haddr1
<32'h8400_0000)
        tempselx = 3'b001; // LED
    else if (Haddr1 >= 32'h8400_0000
&&
        Haddr1 < 32'h8800_0000)
        tempselx = 3'b010; // Timer
    else if (Haddr1 >= 32'h8800_0000
&&
        Haddr1 < 32'h8C00_0000)
        tempselx = 3'b100; // UART
    else
        tempselx = 3'b000;
end
endmodule

```

Apb controller

```

module apb_controller(
    input Hclk, Hresetn, valid,
    Hwritereg,
    input [31:0] Haddr1, Hwdata1,
    input [2:0] tempselx,
    output reg Pwrite, Penable,
    output reg [2:0] Pselx,
    output reg [31:0] Paddr, Pwdata,
    output reg Hreadyout,
    output reg [3:0] current_state //
    To monitor in ILA
);
localparam RESET_ST = 4'd0,
    IDLE    = 4'd1,
    READ    = 4'd2,
    RENABLE = 4'd3,
    WAIT    = 4'd4,
    WRITE   = 4'd5,
    WENABLE = 4'd6;

    reg [3:0] ps, ns;
    reg [31:0] latched_addr,
    latched_wdata;
    reg [2:0] latched_sel;

    // 1. Present State Logic
    (Sequential)
    always @(posedge Hclk or
    negedge Hresetn) begin
        if (!Hresetn) begin
            ps <= RESET_ST;
            latched_addr <= 32'd0;
            latched_wdata <= 32'd0;
            latched_sel <= 3'b000;
        end
        else begin
            ps <= ns;

            if (valid) begin
                latched_addr <= Haddr1;
                latched_sel <= tempselx;
            end

            if (valid)
                latched_wdata <=
Hwdata1;
        end
    end

```

```

    // 2. Next State Logic
    (Combinational)
    always @(*) begin
        case(ps)
            RESET_ST: ns = IDLE;

            IDLE: ns = (!valid) ?
IDLE : (Hwritereg ? WAIT :
READ);
            READ: ns = RENABLE;
            RENABLE: ns = IDLE;
            WAIT: ns = WRITE;
            WRITE: ns = WENABLE;
            WENABLE: ns = IDLE;

            default: ns = IDLE;
        endcase
    end

    // 3. Output Logic
    (Combinational)
    always @(*) begin
        // Default values to prevent
        latches
        Pselx    = 0;
        Penable  = 0;
        Pwrite   = 0;
        Hreadyout = 1;
        Paddr    = latched_addr;
        Pwdata   = latched_wdata;
        current_state = ps;

        case(ps)
            READ: begin
                Pselx = latched_sel;
                Pwrite = 0;
                Hreadyout = 0;
            end

            RENABLE: begin
                Pselx = latched_sel;
                Penable = 1;
                Pwrite = 0;
                Hreadyout = 1;
            end

            WAIT: begin
                Hreadyout = 0;
            end
        end
    end

```

```

        WRITE: begin
            Pselx = latched_sel;
            Pwrite = 1;
            Hreadyout = 0;
        end

        WENABLE: begin
            Pselx = latched_sel;
            Penable = 1;
            Pwrite = 1;
            Hreadyout = 1;
        end
    endcase
end

endmodule

```

LED module

```

module apb_led(input clk, rst, sel,
Pwrite, Penable,
            input [31:0] Pwdata,
            output reg [3:0] led,
            output [31:0] Prdata);
    always @(posedge clk) begin
        if (rst)
            led <= 0;
        else if (sel && Pwrite &&
Penable)
            led <= Pwdata[3:0];
        end
        assign Prdata = {28'd0, led};
    endmodule

```

Timer module

```

module apb_timer(
    input clk, rst, sel, Pwrite, Penable,
    input [31:0] Pwdata,
    output reg tick,
    output [31:0] Prdata
);

    reg [31:0] count, delay;

    always @(posedge clk) begin
        if (rst)
            begin
                count <= 0;
                delay <= 1000000000;
            end
    end

```

```

        tick <= 0;
    end
    else if (sel && Pwrite &&
Penable)
        delay <= Pwdata;
    else if (count >= delay) begin
        count <= 0;
        tick <= 1;
    end
    else begin
        count <= count + 1;
        tick <= 0;
    end
    end
    assign Prdata = delay;
endmodule

```

Pattern module

```

module apb_pattern(
    input clk, rst, tick,
    input [3:0] led_mask,
    output reg [3:0] led_pattern
);

    always @(posedge clk) begin
        if (rst)
            led_pattern <= 4'b0000;
        else if (tick)
            led_pattern <= (led_pattern
== 4'b0000) ? led_mask : 4'b0000;
        end
    endmodule

```

UART module

```

module apb_uart(
    input clk, rst,
    input sel, Pwrite, Penable,
    input [31:0] Pwdata,
    output reg tx,
    output reg baud_tick,
    output [31:0] Prdata
);

    reg [13:0] baud_count;
    reg [3:0] bit_index;
    reg [9:0] shift_reg;
    reg active;
    reg [7:0] tx_fifo [0:7];
    reg [2:0] wr_ptr, rd_ptr;
    reg [3:0] fifo_count;
    initial tx = 1;

```

```

always @(posedge clk)
begin
    if (rst)
        begin
            baud_count <= 0;
            baud_tick <= 0;
        end
    else if (active)
        begin
            if (baud_count ==
10416)
                begin
                    baud_count <= 0;
                    baud_tick <= 1;
                end
            else
                begin
                    baud_count <=
baud_count + 1;
                    baud_tick <= 0;
                end
            end
        end
    else
        begin
            baud_count <= 0;
            baud_tick <= 0;
        end
    end

always @(posedge clk)
begin
    if (rst)
        begin
            tx <= 1;
            active <= 0;
            bit_index <= 0;
            shift_reg <=
10'b1111111111;
            wr_ptr <= 0;
            rd_ptr <= 0;
            fifo_count <= 0;
        end
    else begin
        if (sel && Pwrite &&
Penable) begin
            if (!active && (fifo_count
== 0)) begin
                // Start the very first
byte immediately.
                shift_reg <= {1'b1,

```

```

Pwdata[7:0], 1'b0};
            active <= 1;
            bit_index <= 0;
            tx <= 1;
        end
        else if (fifo_count < 8)
            begin
                // Queue later bytes
while a transfer is already in
progress.
                tx_fifo[wr_ptr] <=
Pwdata[7:0];
                wr_ptr <= wr_ptr + 1;
                fifo_count <=
fifo_count + 1;
            end
        end
        if (active && baud_tick)
            begin
                tx <= shift_reg[0];
                shift_reg <= {1'b1,
shift_reg[9:1]};
                if (bit_index == 9) begin
                    active <= 0;
                    bit_index <= 0;
                    tx <= 1;
                end
                else begin
                    bit_index <= bit_index
+ 1;
                end
            end
        if (!active && (fifo_count !=
0)) begin
            shift_reg <= {1'b1,
tx_fifo[rd_ptr], 1'b0};
            active <= 1;
            bit_index <= 0;
            rd_ptr <= rd_ptr + 1;
            fifo_count <= fifo_count -
1;
            tx <= 1;
        end
    end
end
end
assign Prdata = {24'd0,
tx_fifo[rd_ptr]};
endmodule

```

7.segment display module

```

module display(
    input clk,    input rst,
    input op_write, input [2:0]
peripheral,
    input [7:0] data_byte,    output
reg [6:0] seg,
    output reg [3:0] an
);
    reg [15:0] refresh_count;
    reg [1:0] digit_sel;

    wire [3:0] nibble_hi =
data_byte[7:4];
    wire [3:0] nibble_lo =
data_byte[3:0];

    localparam OP_R = 2'b00;
    localparam OP_W = 2'b01;
    always @(posedge clk) begin
        if (rst)
            refresh_count <= 0;
        else
            refresh_count <=
refresh_count + 1;
        end

    always @(*) begin
        digit_sel =
refresh_count[15:14];
        an = 4'b1111;
        seg = 7'b1111111;

        case (digit_sel)
            2'b00: begin
                an = 4'b1110;
                seg =
hex_to_seg(nibble_lo);
            end
            2'b01: begin
                an = 4'b1101;
                seg =
hex_to_seg(nibble_hi);
            end
            2'b10: begin
                an = 4'b1011;
                seg =
peripheral_to_seg(peripheral);
            end
            2'b11: begin

```

```

                an = 4'b0111;
                seg = op_to_seg(op_write
? OP_W : OP_R);
            end
        endcase
    end
    function [6:0] hex_to_seg;
        input [3:0] value;
        begin
            case (value)
                4'h0: hex_to_seg =
7'b1000000;
                4'h1: hex_to_seg =
7'b1111001;
                4'h2: hex_to_seg =
7'b0100100;
                4'h3: hex_to_seg =
7'b0110000;
                4'h4: hex_to_seg =
7'b0011001;
                4'h5: hex_to_seg =
7'b0010010;
                4'h6: hex_to_seg =
7'b0000010;
                4'h7: hex_to_seg =
7'b1111000;
                4'h8: hex_to_seg =
7'b0000000;
                4'h9: hex_to_seg =
7'b0010000;
                4'hA: hex_to_seg =
7'b0001000;
                4'hB: hex_to_seg =
7'b0000011;
                4'hC: hex_to_seg =
7'b1000110;
                4'hD: hex_to_seg =
7'b0100001;
                4'hE: hex_to_seg =
7'b0000110;
                default: hex_to_seg =
7'b0001110;
            endcase
        end
    endfunction
    function [6:0] peripheral_to_seg;
        input [2:0] value;
        begin
            case (value)
                3'b001: peripheral_to_seg

```

```

= 7'b1111001; // 1 = LED
    3'b010: peripheral_to_seg
= 7'b0100100; // 2 = TIMER
    3'b100: peripheral_to_seg
= 7'b0110000; // 3 = UART
    default: peripheral_to_seg
= 7'b1111111; // blank
    endcase
end
endfunction
function [6:0] op_to_seg;
    input [1:0] value;
    begin
        case (value)
            OP_W: op_to_seg =
7'b1101010; // approximate 'W'
            default: op_to_seg =
7'b0101111; // approximate 'r'
            endcase
        end
    endfunction
endmodule

```

constranist

```

## CLOCK SIGNAL (100 MHz)##
set_property PACKAGE_PIN W5
[get_ports Hclk]
set_property IOSTANDARD
LVCMOS33 [get_ports Hclk]
create_clock -name sys_clk -period
10.000 [get_ports Hclk]

```

```

## RESET (Center Push Button -
ACTIVE HIGH)##
set_property PACKAGE_PIN U18
[get_ports Hresetn]
set_property IOSTANDARD
LVCMOS33 [get_ports Hresetn]

```

```

## LEDs (4-bit output)##
set_property PACKAGE_PIN U16
[get_ports {LED[0]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {LED[0]}]

```

```

set_property PACKAGE_PIN E19

```

```

[get_ports {LED[1]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {LED[1]}]

```

```

set_property PACKAGE_PIN U19
[get_ports {LED[2]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {LED[2]}]

```

```

set_property PACKAGE_PIN V19
[get_ports {LED[3]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {LED[3]}]

```

```

## UART TX (USB-UART via
FTDI)##
set_property PACKAGE_PIN A18
[get_ports tx]
set_property IOSTANDARD
LVCMOS33 [get_ports tx]

```

```

## Seven-Segment Display ##
set_property PACKAGE_PIN W7
[get_ports {seg[0]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {seg[0]}]

```

```

set_property PACKAGE_PIN W6
[get_ports {seg[1]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {seg[1]}]

```

```

set_property PACKAGE_PIN U8
[get_ports {seg[2]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {seg[2]}]

```

```

set_property PACKAGE_PIN V8
[get_ports {seg[3]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {seg[3]}]

```

```

set_property PACKAGE_PIN U5
[get_ports {seg[4]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {seg[4]}]

```

```

set_property PACKAGE_PIN V5
[get_ports {seg[5]}]

```

```

set_property IOSTANDARD
LVCMOS33 [get_ports {seg[5]}]

set_property PACKAGE_PIN U7
[get_ports {seg[6]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {seg[6]}]

set_property PACKAGE_PIN U2
[get_ports {an[0]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {an[0]}]

set_property PACKAGE_PIN U4
[get_ports {an[1]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {an[1]}]

set_property PACKAGE_PIN V4
[get_ports {an[2]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {an[2]}]

set_property PACKAGE_PIN W4
[get_ports {an[3]}]
set_property IOSTANDARD
LVCMOS33 [get_ports {an[3]}]

## Configuration bank settings for
Basys 3 ##
set_property CFGBVS VCCO
[current_design]
set_property CONFIG_VOLTAGE
3.3 [current_design]

## BITSTREAM SETTINGS
(Basys 3 Flash) ##
set_property
BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property
BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE
SPIx4 [current_design]

```

Testbench Code

```

module ahb_apb_tb;

    reg Hclk;

    reg Hresetn;

    wire [3:0] LED;

    wire tx;

    wire [6:0] seg;

    wire [3:0] an;

    wire Hwrite_tb = DUT.Hwrite_gen;

    wire [1:0] Htrans_tb = DUT.Htrans_gen;

    wire [31:0] Haddr_tb = DUT.Haddr_gen;

    wire [31:0] Hwdata_tb =
DUT.Hwdata_gen;

    wire valid_tb= DUT.valid;

    wire [31:0] Haddr1_tb = DUT.Haddr1;

    wire [31:0] Hwdata1_tb = DUT.Hwdata1;

    wire Hwritereg_tb = DUT.Hwritereg;

    wire [3:0] state_tb=
DUT.current_fsm_state;

    wire [2:0] Pselx_tb = DUT.Pselx;

    wire Penable_tb = DUT.Penable;

    wire Pwrite_tb = DUT.Pwrite;

    wire [31:0] Paddr_tb = DUT.Paddr;

    wire [31:0] Pwdata_tb =
DUT.Pwdata;

    wire [31:0] Prdata_tb = DUT.Prdata;

    wire [31:0] transfer_data_tb =
DUT.transfer_data;

```